Optimization Techniques
Electrical Computer and Engineering Dept.
Aristotle University of Thessaloniki
7th Semester - November 2025

Anastasia Loukeri
Student ID:10975
aloukeri@ece.auth.gr

General Problem Description

Goal: The objective is to find the minimum of a given multivariable function $f(x)$ without constraints and with utility of derivatives. More specifically, the search algorithms that will be studied are:

- **Steepest Descent Method**
- **Newton's Method**
- **Levenberg-Marquardt Method**

The algorithms we will use are based on the idea of iterative descent, based on which we start from a point $x_0 \in R^n$ and successively generate vectors $x_1, x_2, \ldots$ such that $f(x_{k+1}) < f(x_k), k = 1, 2, \ldots$. The objective function we will study is:

$$f(x, y) = x^3 e^{-x^2-y^4}$$

Requirement 1 - Mathematical Analysis

1. Gradient Calculation

To apply descent methods, we first calculate the gradient vector $\nabla f(x, y)$. The partial derivatives are:

$$\begin{aligned}
\frac{\partial f}{\partial x} &= \frac{\partial}{\partial x}\left(x^3 e^{-x^2-y^4}\right) \\
&= 3x^2 e^{-x^2-y^4} + x^3(-2x)e^{-x^2-y^4} \\
&= e^{-x^2-y^4}(3x^2 - 2x^4) \\
&= x^2 e^{-x^2-y^4}(3 - 2x^2)
\end{aligned}$$

$$\begin{aligned}
\frac{\partial f}{\partial y} &= \frac{\partial}{\partial y}\left(x^3 e^{-x^2-y^4}\right) \\
&= x^3 e^{-x^2}(-4y^3 e^{-y^4}) \\
&= -4x^3 y^3 e^{-x^2-y^4}
\end{aligned}$$

Thus, the gradient is:

$$\nabla f(x, y) = \begin{pmatrix} x^2(3 - 2x^2)e^{-x^2-y^4} \\ -4x^3 y^3 e^{-x^2-y^4} \end{pmatrix}$$

2. Critical Points

We find the critical points by setting $\nabla f(x, y) = \mathbf{0}$. Since $e^{-x^2-y^4} \neq 0$ for all real numbers, we can simplify the system:

$$\begin{cases} x^2(3 - 2x^2) = 0 & (1) \\ -4x^3y^3 = 0 & (2) \end{cases}$$

From equation (1), we have two cases for x:

$$x = 0 \quad \text{or} \quad 3 - 2x^2 = 0 \Rightarrow x^2 = \frac{3}{2} \Rightarrow x = \pm\sqrt{1.5}$$

Case A: $x = 0$

Substituting into equation (2), $-4(0)^3y^3 = 0$, which is true for any y. Therefore, all points on the y-axis $(0, y)$ are critical points. The function value is $f(0, y) = 0$.

Case B: $x \neq 0$

From equation (1), we must have $x = \pm\sqrt{1.5}$. From equation (2), since $x \neq 0$, we must have $y^3 = 0 \Rightarrow y = 0$. This gives two critical points:

$$P_1(\sqrt{1.5}, 0) \quad \text{and} \quad P_2(-\sqrt{1.5}, 0)$$

3. Finding the Minimum

To find the global minimum, we evaluate the function at the critical points and analyze the asymptotic behavior.

- For any point $(0, y)$, $f(0, y) = 0$.
- At $P_1(\sqrt{1.5}, 0)$:
$$f(\sqrt{1.5}, 0) = (\sqrt{1.5})^3e^{-1.5} > 0$$
- At $P_2(-\sqrt{1.5}, 0)$:
$$f(-\sqrt{1.5}, 0) = (-\sqrt{1.5})^3e^{-1.5} = -1.5\sqrt{1.5}e^{-1.5} \approx -0.41$$

Analysis: The function $f(x, y) = x^3e^{-x^2-y^4}$ tends to 0 as $x, y \to \pm\infty$ due to the exponential term decaying faster than the polynomial growth. Since the function takes negative values (when $x < 0$), the global minimum must occur at a critical point with a negative value.

The only critical point with a negative value is P_2.

Conclusion: The global minimum is located at:

$$(x, y) = \left(-\sqrt{1.5}, 0\right) = \left(-\frac{\sqrt{6}}{2}, 0\right)$$

With a minimum value of:

$$f_{min} = -\left(\frac{3}{2}\right)^{\frac{3}{2}} e^{-\frac{3}{2}} \approx -0.413$$

This analysis is visually confirmed by the plot of the function.

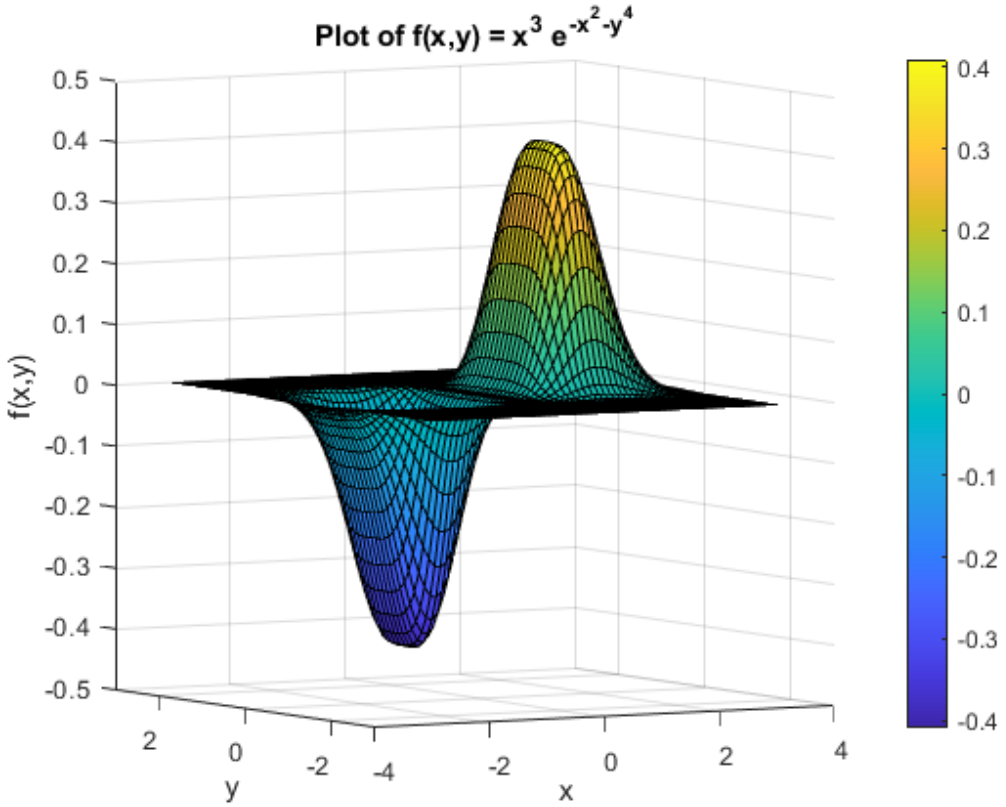


Figure 1: **3D Surface Plot of the Objective Function** $f(x, y) = x^3 e^{-x^2-y^4}$**.**

The provided 3D surface plot offers a powerful visual confirmation of the analytical results derived above. The visualization maps the function $f(x, y)$ over a domain roughly spanning $[-4, 4]$.
Topological Features: On the left side of the plot (negative x-axis), there is the global minimum colored in deep blue. The color bar confirms the depth of this valley (**Global Minimum** corresponds to approximately -0.4, matching our calculated $f_{min} \approx -0.413$.
Symmetrically opposite on the positive x-axis, there is a sharp peak (**Global Maximum**) colored in red, reaching a height of approximately $+0.4$.
The "Squeezed" Y-Axis: A notable feature of the plot is how narrow the peak and valley are along the y-axis. This is mathematically explained by the y^4 term in the exponent. The function e^{-y^4} decays much faster than a standard Gaussian (e^{-y^2}), resulting in a "flattening" effect that confines the non-zero values to a narrow strip near $y = 0$.
The Flat Saddle Line: The plot is completely flat and green (value 0) along the line $x = 0$. This visualizes our finding that the gradient is zero along the entire y-axis.

Mathematical Classification of Critical Points

Solving the equation $\nabla f = 0$, we obtain the following critical points:

$$(0, a) \text{ for any } a \in R, \quad (1.2247, 0), \quad \text{and} \quad (-1.2247, 0).$$

Both from the analysis of the function f and from the graphical representations, we conclude that the two points $(1.2247, 0)$ and $(-1.2247, 0)$ are the locations where the **global maximum** and **global minimum** exist, respectively.

In contrast, the points $(0, a)$ constitute **saddle points** (or flat inflection points). This is confirmed by the Hessian matrix $\nabla^2 f(x, y)$, which contains the second-order partial derivatives:

$$H(x, y) = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix}$$

$$H(x, y) = e^{-x^2-y^4} \begin{pmatrix} x(4x^4 - 14x^2 + 6) & -4x^2y^3(3 - 2x^2) \\ -4x^2y^3(3 - 2x^2) & -4x^3y^2(3 - 4y^4) \end{pmatrix}$$

Evaluating the Hessian at the points $(0, a)$, we observe that the matrix is the zero matrix (due to the term x^2 and x^3 in the derivatives vanishing at $x = 0$). Consequently, the Hessian $\nabla^2 f(0, a)$ is **neither positive definite** (which would characterize local minima) **nor negative definite** (which would characterize local maxima).

The above mathematical analysis helps us better understand the results of the algorithms for different initial starting points.

Requirement 2 - Steepest Descent

In this method, to select the next point, we define $x_{k+1} = x_k + \gamma_k \cdot d_k$, where $d_k = -\nabla f(x_k)$. Our choice relies on the fact that the inner product $\nabla f^T(x_k) \cdot d_k$ is minimized when $d_k = -\frac{\nabla f(x_k)}{|\nabla f(x_k)|}$. With an appropriate choice of γ_k at each iteration, we can calculate the next point x_{k+1}.

For the implementation of the method, we followed Algorithm 5.2.1, page 121 of the book. Our deviation from this algorithm lies in the rule by which γ_k is selected, as in our implementation we have three different ways of making the selection, which we present below together with the results regarding the convergence (or non-convergence) of the algorithm.

Application to the Objective Function

We apply this method to our function:

$$f(x, y) = x^3 e^{-x^2-y^4}$$

First, recall the gradient vector derived in the previous section:

$$\nabla f(x, y) = e^{-x^2-y^4} \begin{pmatrix} x^2(3 - 2x^2) \\ -4x^3y^3 \end{pmatrix}$$

We analyze the behavior of the Steepest Descent method for the three given initial points (x_0, y_0) and different choices of step size γ_k.

Constant step size γ_k

The steepest descent method step size is crucial for controlling the algorithm's progress, balancing speed with stability. In this section γ_k is a fixed but tuned parameter, but a small value will be eventually chosen as a practical alternative. If the step size is too small, convergence is slow; if it's too large, the algorithm may overshoot the minimum or become unstable.

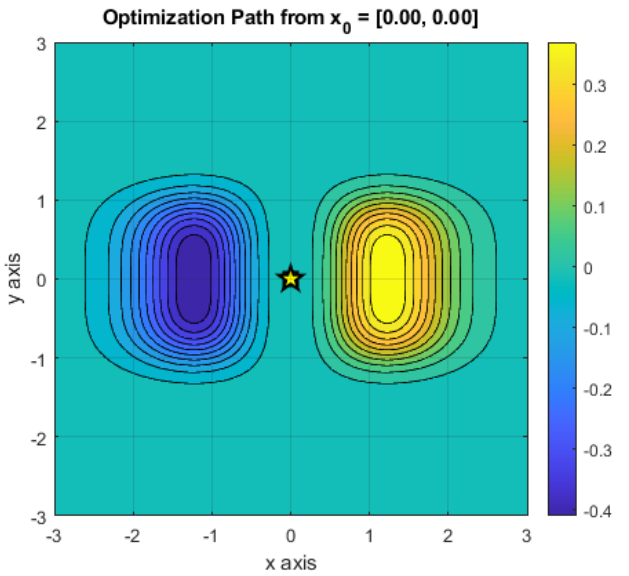
Case 1: $\gamma_k = 0.1$

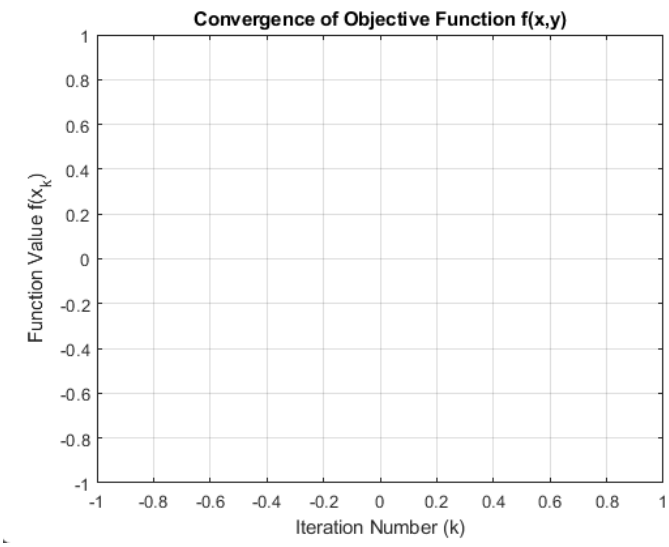
Initial point $(0, 0)$

Substituting $x = 0, y = 0$ into the gradient:

$$\nabla f(0,0) = e^0 \begin{pmatrix} 0(3-0) \\ -4(0)(0) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Result: Since the gradient is the zero vector, the algorithm terminates immediately. The point $(0, 0)$ is a critical point (specifically, part of the saddle line along the y-axis).This is attributed to the fact that the entire locus of points $(0, y)$ constitutes a set of stationary points where the gradient vector vanishes ($\nabla f = \mathbf{0}$). Consequently, regardless of the specific search algorithm or the step-size selection strategy (γ_k) employed, it is impossible to generate a descent direction from this coordinate. The following 2-D plot demonstrates that.





Initial Point $(-1, -1)$

Substituting $x = -1, y = -1$:

$$\text{Exponential term: } e^{-(-1)^2-(-1)^4} = e^{-1-1} = e^{-2}$$

$$\frac{\partial f}{\partial x} = e^{-2}(-1)^2(3 - 2(-1)^2) = e^{-2}(1)(1) = e^{-2}$$

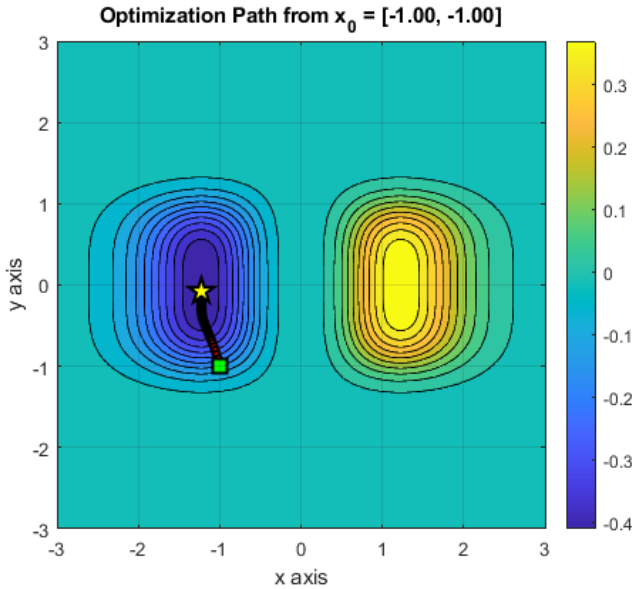
$$\frac{\partial f}{\partial y} = -4(-1)^3(-1)^3 e^{-2} = -4(-1)(-1)e^{-2} = -4e^{-2}$$

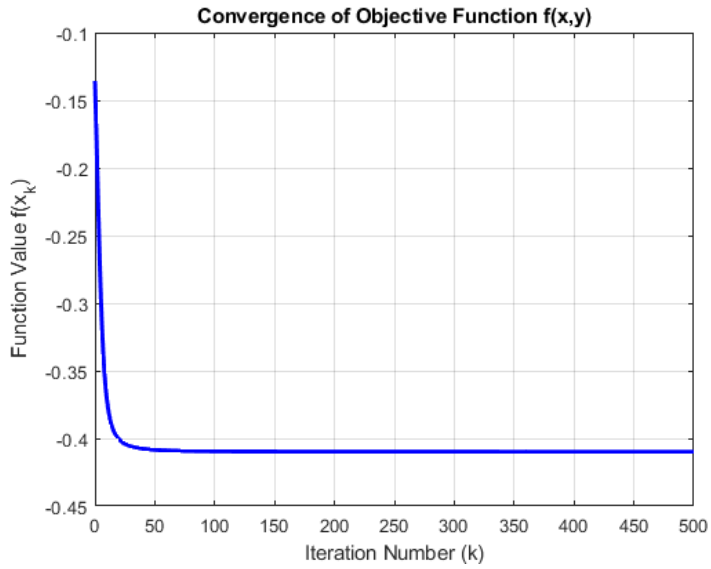
$$\nabla f(-1,-1) = \begin{pmatrix} e^{-2} \\ -4e^{-2} \end{pmatrix} \approx \begin{pmatrix} 0.135 \\ -0.541 \end{pmatrix}$$

The direction of descent is $d_0 = -\nabla f$:

$$d_0 = \begin{pmatrix} -e^{-2} \\ 4e^{-2} \end{pmatrix}$$

Result: The algorithm will move in the negative x direction (towards the global minimum at $x \approx -1.22$) and the positive y direction (towards $y = 0$). This point is well-positioned to converge to the global minimum.The starting point $(-1,-1)$ is already located on the "slope" leading down to the valley. The gradient here points clearly towards the minimum. The path is smooth and curves directly into the center of the blue region.The function value drops very quickly in the first few iterations (steep slope) and then settles at the minimum value of ≈ -0.41. This is a "monotonic decrease," meaning it gets better at every single step.





Initial Point $(1,1)$

Substituting $x = 1, y = 1$:

$$\text{Exponential term: } e^{-1^2-1^4} = e^{-2}$$

$$\frac{\partial f}{\partial x} = e^{-2}(1)^2(3-2(1)^2) = e^{-2}(1)(1) = e^{-2}$$

$$\frac{\partial f}{\partial y} = -4(1)^3(1)^3e^{-2} = -4(1)(1)e^{-2} = -4e^{-2}$$

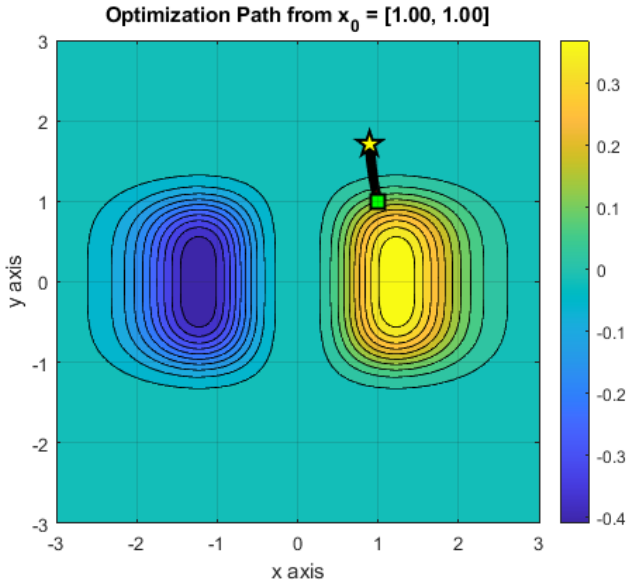
$$\nabla f(1,1) = \begin{pmatrix} e^{-2} \\ -4e^{-2} \end{pmatrix}$$

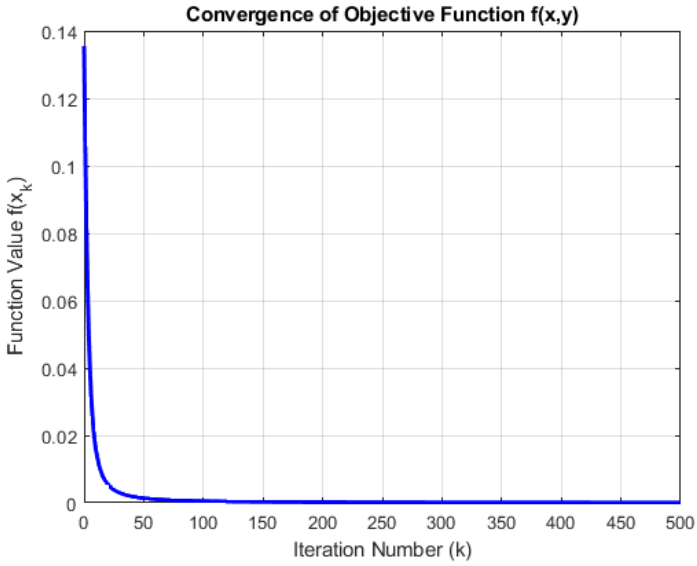
The direction of descent is $d_0 = -\nabla f$:

$$d_0 = \begin{pmatrix} -e^{-2} \\ 4e^{-2} \end{pmatrix}$$

Result: The algorithm moves in the negative x direction and positive y direction.By moving towards larger y values (to escape the local maximum hill), the point enters the region where

the term e^{-y^4} dominates.As y increases, the function value $f(x, y)$ becomes incredibly small (approaching 0) as shown in the second figure. Consequently, the gradient ∇f also becomes infinitesimally small. The algorithm terminated because the gradient magnitude dropped below the set tolerance.





Case 2: $\gamma_k = 2$

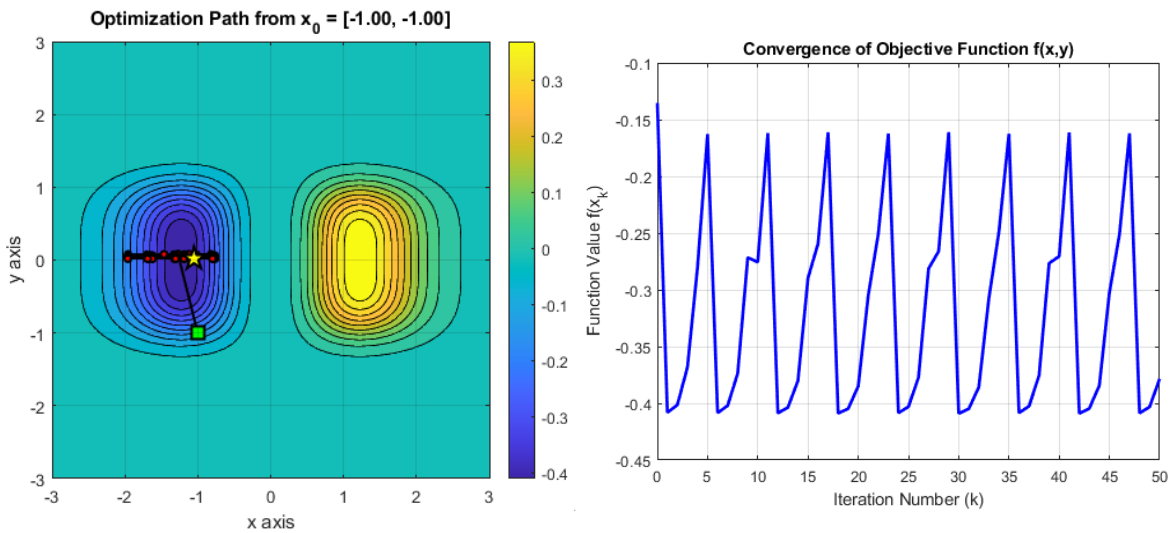
The choice of step size γ is critical.This specific case demonstrates the dangers of a "too large" step size. Using a large, fixed step size $\gamma_k = 2$ leads to instability as we will represent.

Initial Point $(-1, -1)$

When starting near the solution basin, $\gamma = 2$ causes the "Zig-Zag" phenomenon. The Steepest Descent update rule is:

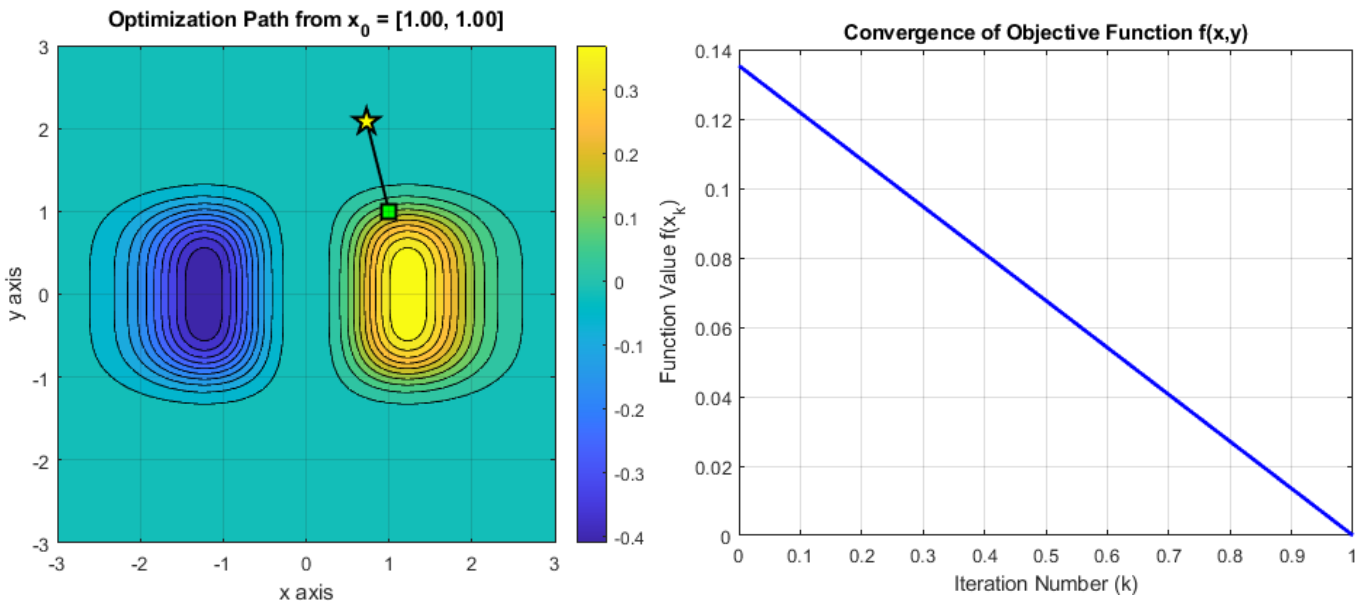
$$\mathbf{x}_{k+1} = \mathbf{x}_k - \gamma \nabla f(\mathbf{x}_k)$$

In a narrow valley (like the one surrounding our global minimum), the behavior unfolds as follows:
Position A (Left Wall): The algorithm is at $\mathbf{x}_k$ on the "left" slope of the valley. The gradient ∇f points "downhill" to the right.The Overshoot: The gradient magnitude $|\nabla f|$ is moderate, but the step size $\gamma = 2$ is excessive. The computed step $\gamma \nabla f$ is larger than the width of the valley floor.
Position B (Right Wall): The algorithm jumps completely over the minimum and lands on the "right" slope at $\mathbf{x}_{k+1}$. Crucially, because it jumped so far, it landed at a point where the elevation $f(\mathbf{x}_{k+1})$ is roughly the same (or higher) than where it started.The Reversal: At Position B, the gradient now points "downhill" to the left.The Loop: The algorithm applies the same large jump in the opposite direction, landing back near Position A. This creates a Limit Cycle, where the point bounces back and forth forever like a ping-pong ball.To summarize, the algorithm steps over the minimum to the opposite slope. The gradient direction reverses, and the next step overshoots back to the original side. As seen in the convergence plots, the function value oscillates between two bounds (e.g., -0.4 and -0.16) and never settles. The system enters a stable limit cycle, thus becomes trapped in an endless loop, bouncing back and forth across the minimum without ever settling down. failing to converge to the precise minimum $\mathbf{x}^*$.



Initial Point $(1, 1)$

The algorithm takes a step in the direction of the negative gradient. However, because $\gamma = 2$ is a large multiplier, the algorithm "overshoots" significantly. This large step propels the point from $y = 1$ directly to approximately $y \approx 2.1$, where the term e^{-y^4} is astronomically small and the function value becomes effectively zero as proved in the following figures. The algorithm terminates in the first iteration.



Influence of the Step Size Parameter γ_k

The selection of the parameter γ (step size) critically influences both the convergence rate and the stability of the Steepest Descent algorithm. Based on our experiments, we observe the following distinct behaviors:

1. **For initial points within the basin of attraction (e.g., $[-1, -1]$):**

- With a **small or adaptive** γ, the algorithm converges smoothly and monotonically to the global minimum ($f \approx -0.41$).
- However, with a **large** γ (e.g., $\gamma = 2$), the step becomes excessive relative to the curvature of the valley. This leads to significant deviations and results in *oscillatory instability*, where the algorithm enters a limit cycle jumping back and forth across the minimum without ever settling.

2. **For initial points on the opposing slope (e.g.,** $[1, 1]$**):**

   - With a **small** γ, the algorithm faithfully follows the gradient, which unfortunately directs the path towards the asymptotic "flat" region where $f \to 0$. It fails to find the global minimum because the starting point lies outside the correct basin of attraction.
   - With a **large** γ, the algorithm suffers from *overshooting*. It performs a massive jump that propels the point deep into the region where y is large. In this region, the gradient effectively vanishes ($|\nabla f| \approx 0$), causing the algorithm to terminate prematurely at a point where the function value is practically zero, erroneously signalling convergence.

3. **For the initial point** $[0, 0]$**:**

   - Since this point lies on a line of critical points where the gradient is exactly zero ($\nabla f = \mathbf{0}$), the algorithm terminates immediately after the first iteration, regardless of the choice of γ. No descent is possible from this stationary point using first-order derivatives.

Conclusion

The above analysis underscores the critical importance of selecting the correct parameter γ. A fixed, arbitrary γ often leads to either slow convergence, oscillation, or premature termination in flat regions. Consequently, there is a clear necessity for **dynamic step size selection** (such as the Armijo rule as we will demonstrate later) which adjusts γ_k at every iteration to guarantee a decrease in the objective function and ensure stability.

Optimal Step Size γ_k minimizing $f(x_k + \gamma_k \cdot d_k)$

In each iteration of the Steepest Descent algorithm—after determining the search direction $d_k = -\nabla f(x_k)$—we proceed to calculate the optimal step size γ_k. This process, known as **Exact Line Search**, involves finding the value of γ that minimizes the scalar objective function along the search direction:

$$g(\gamma) = f(x_k + \gamma \cdot d_k)$$

A necessary condition for γ^* to be a local minimizer of $g(\gamma)$ is that the first derivative of g with respect to γ must be zero. By applying the chain rule, this condition is expressed as:

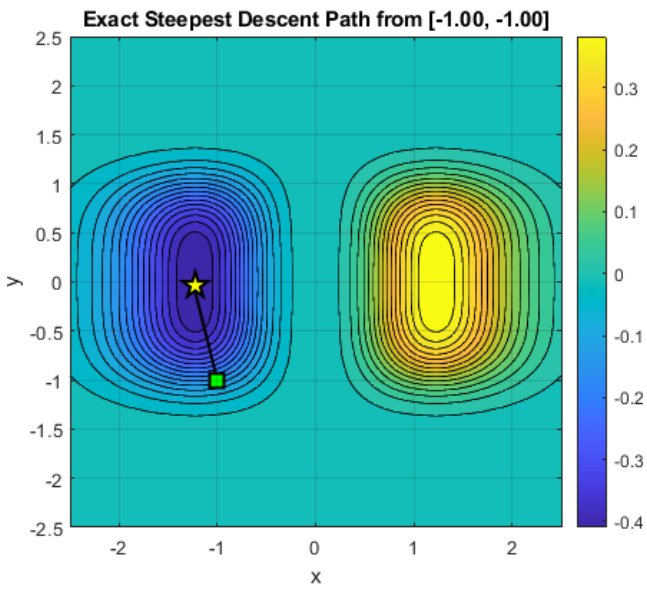
$$g'(\gamma) = \nabla f(x_k + \gamma \cdot d_k)^T \cdot d_k = 0 \tag{1}$$

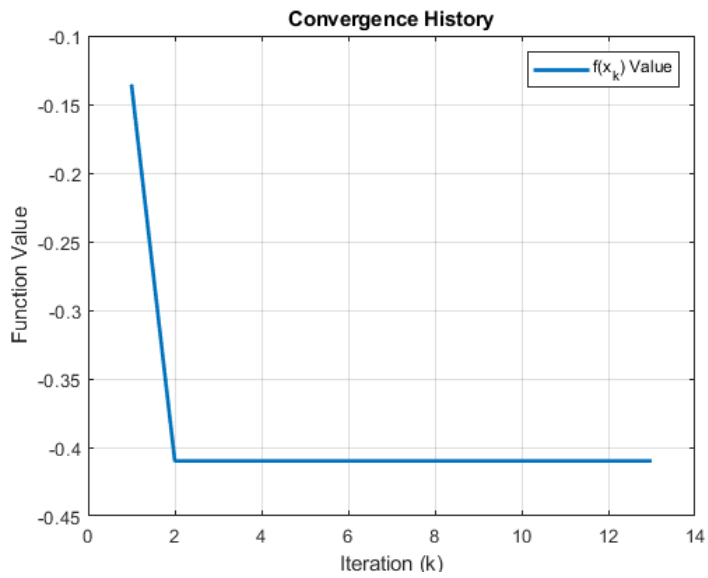
Consequently, the optimization problem transforms into a root-finding problem for the function $g'(\gamma)$ where γ is chosen to belong in the $(0, 2)$ interval.

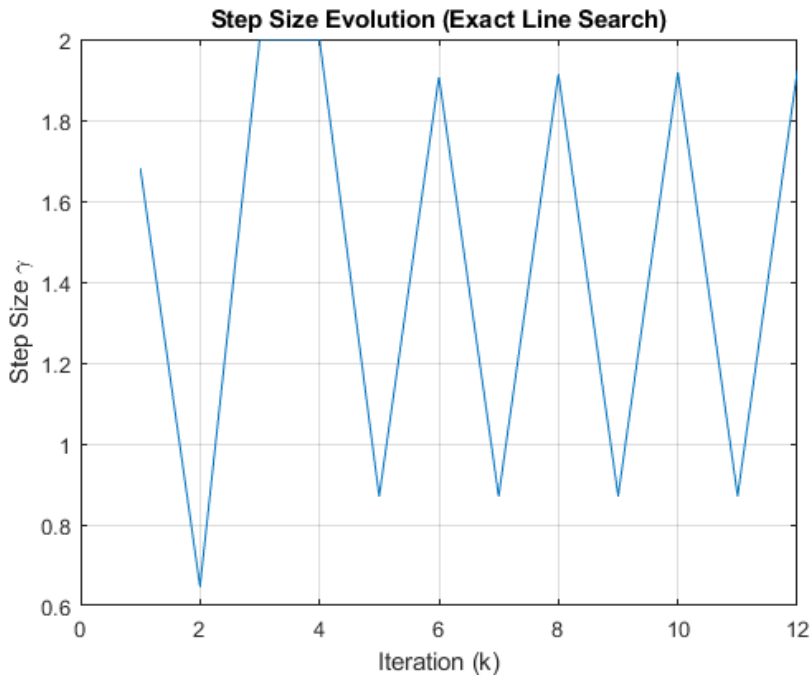
To solve this, we employ the **Bisection Method with Derivative** implemented in the first project.

Initial point $(-1, -1)$

The Steepest Descent algorithm, employing the derivative-based Bisection method for Exact Line Search from $\mathbf{x}_0 = (-1, -1)$, converges effectively to the global minimum. The objective function value decreases monotonically, reaching the theoretical minimum of $f^* \approx -0.413$ within the first 10 iterations. The computed step size γ_k oscillates between approximately 0.9 and 2.0. This fluctuation occurs because Exact Line Search imposes orthogonality on consecutive search directions ($\nabla f_{k+1}^T \nabla f_k = 0$). Due to the narrow geometry of the objective function's valley, this orthogonality results in a zig-zag trajectory. The algorithm automatically adjusts the step magnitude at each iteration to traverse this path efficiently, ensuring the steepest possible descent at every step.

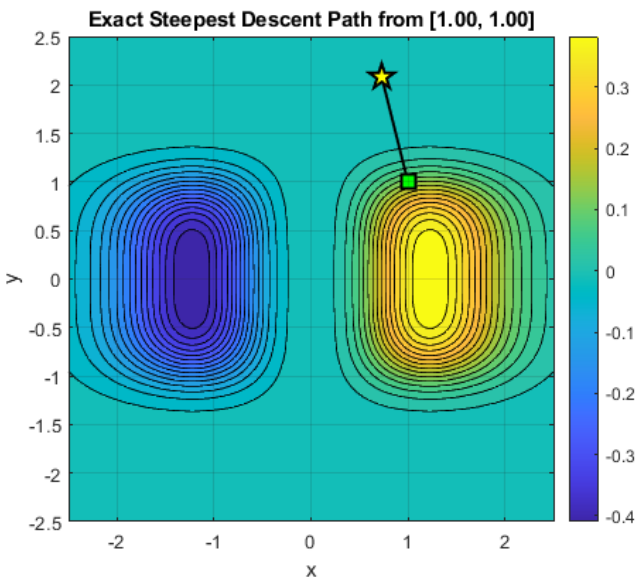


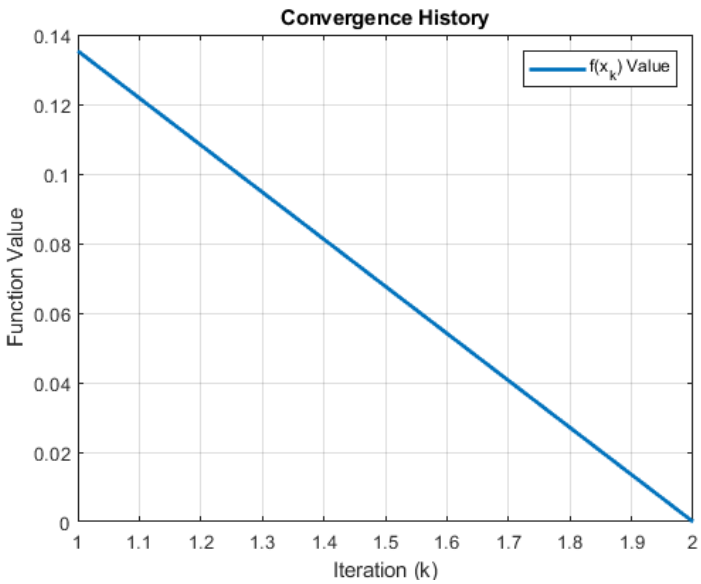




Initial Point $(1, 1)$

At $(1, 1)$, the gradient points to the direction where y increases and x decreases as shown with the calculation of ∇f in that point. The algorithm correctly calculates that to reduce the function value immediately, it must move away from the peak where the Global Maximum is located.Then the Bisection method searches along this direction to find the lowest possible function value to choose the suitable γ_k. Along this specific line, the function does not cross into the deep negative valley; instead, it slopes down continuously until it asymptotically approaches zero as $y \to \infty$. Once the point lands in this flat region at the second iteration, the gradient becomes effectively zero ($\nabla f \approx \mathbf{0}$). The algorithm assumes it has found a minimum and terminates, because this flat region consists of the saddle points $(0, a)$ where $\nabla f \approx \mathbf{0}$. It has minimized the function locally to 0, but failed to find the global minimum of -0.41.





Step Size γ_k updated by Armijo's Rule

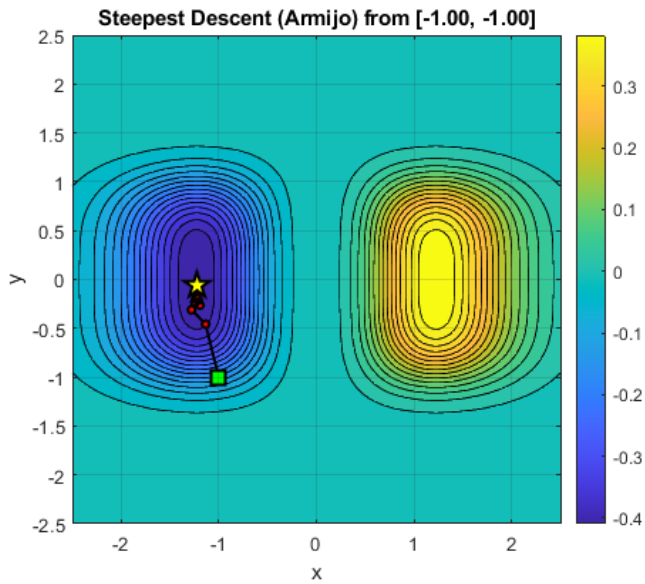
The Armijo Rule is an iterative method used to determine a step size γ_k that guarantees a sufficient decrease in the objective function at each iteration, rather than solving for the exact minimum along the search direction.
This strategy enforces the Sufficient Decrease Condition:

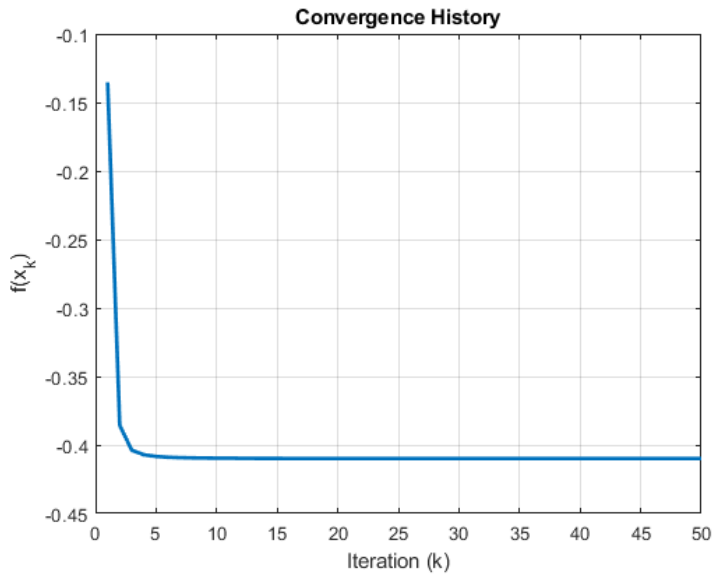
$$f(\mathbf{x}_k + \gamma_k \mathbf{d}_k) \leq f(\mathbf{x}_k) + \sigma \gamma_k \nabla f(\mathbf{x}_k)^T \mathbf{d}_k \tag{2}$$

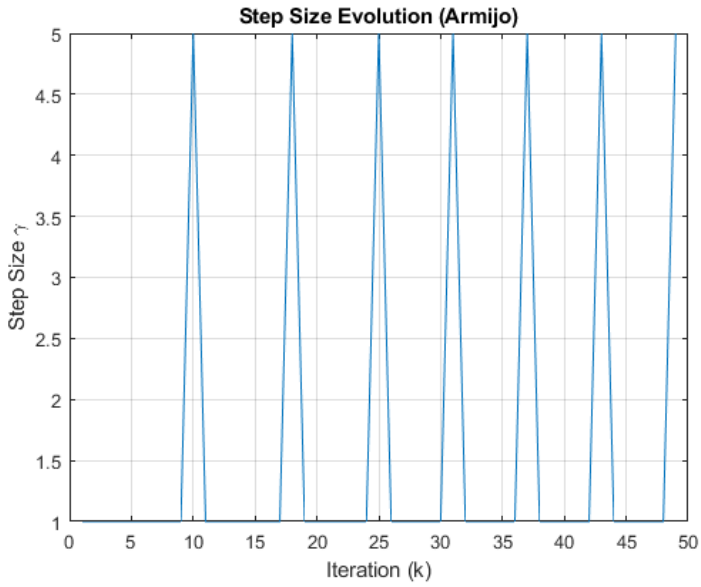
where $\sigma \in (0, 1)$ is a relaxation parameter that defines the minimum acceptable drop in the function value relative to the predicted decrease based on the local gradient. For our project we assigned $\sigma = 10^{-4}$. The algorithm initializes with a trial step length s and iteratively applies a reduction factor $\beta \in (0, 1)$ until the inequality is satisfied. For our project we assigned β=0.2 and an initial large step size value of γ_k=5.
In regions of high curvature, such as the steep walls of the global minimum's basin, the condition rejects large steps that would violate the inequality, forcing the algorithm to "backtrack" to a smaller, stable step size. Conversely, in flatter regions where the gradient magnitude is small, the condition is more easily met, allowing for larger steps. This dynamic adjustment ensures that the algorithm consistently progresses towards the minimum while maintaining strict theoretical stability guarantees.

Initial Point $(-1, -1)$

The implementation of the Steepest Descent method utilizing the Armijo Rule from the initial point $\mathbf{x}_0 = (-1, -1)$ results in a stable and efficient convergence to the global minimum. The convergence history indicates a rapid, monotonic reduction in the objective function value, stabilizing at the minimum $f^* \approx -0.413$ within the first few iterations.
The Step Size Evolution plot shows the adaptive nature of **Armijo Rule**.The spikes around γ_k=5 represent the iterations where the local curvature of the function was low and the algorithm attempted a large initial step size and because **Armijo's Condition** was met, the step was accepted. Other iterations reach the baseline γ_k=1 where the steepness of the valley walls violated the sufficient decrease condition for the large trial step. Consequently, the algorithm backtracked, applying the reduction factor β—until a safe step size was found.

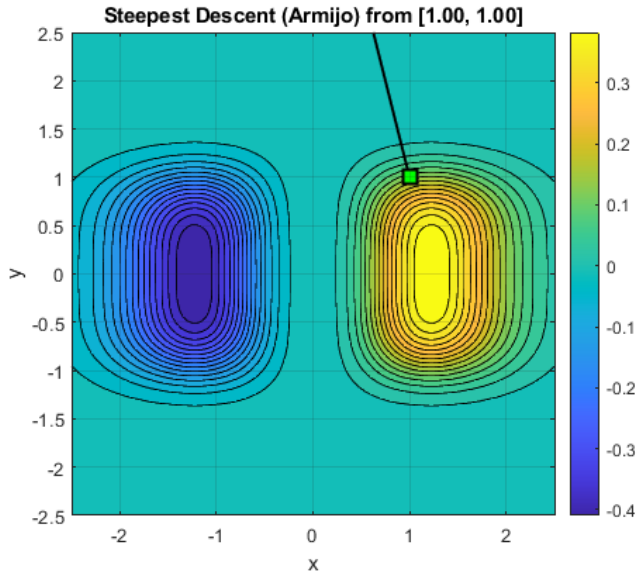


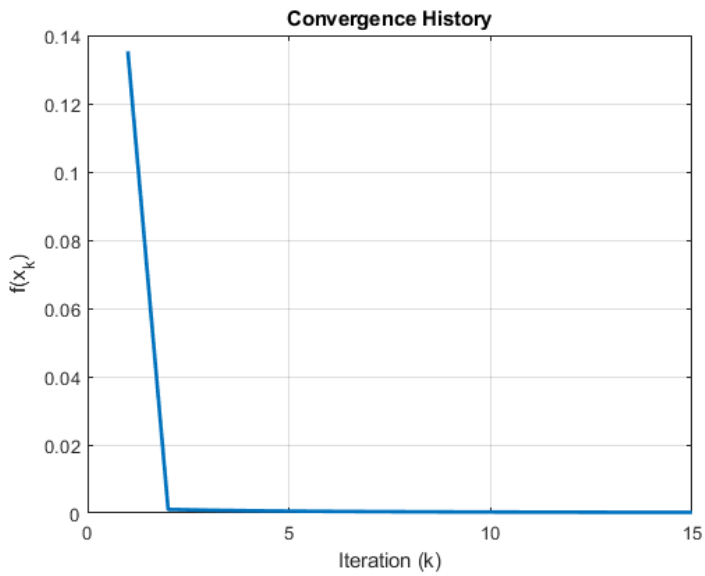




Initial Point $(1, 1)$

The application of the Steepest Descent method with the Armijo Rule from the initial point $\mathbf{x}_0 = (1, 1)$ results in convergence to the trivial asymptotic limit of the function ($f \to 0$), failing to locate the global minimum ($f \approx -0.41$). This trajectory exhibits a distinct "escape" behavior, where the descent path extends rapidly beyond the effective range of the local maxima and minima. As observed in the figure, the algorithm generates a search vector that propels the iterator immediately out of the visible domain ($y > 2.5$). At $(1, 1)$, the negative gradient directs the search towards increasing values of y to reduce the function value. The objective function contains the term e^{-y^4}, which induces extremely rapid decay. Consequently, even a very large trial step size (e.g., $\gamma_0 \gg 1$) results in a new function value that is effectively zero. Since this represents a substantial decrease from the initial value of $f \approx 0.135$, the Armijo inequality is satisfied without the need for backtracking and within just 2 iterations. The Armijo rule successfully ensured a decrease in the objective function without instability.In this region, the gradient magnitude vanishes ($\nabla f \approx \mathbf{0}$), causing the algorithm to terminate. Thus, the steepest descent path from this specific initialization converges to the trivial asymptotic limit rather than the global minimum.



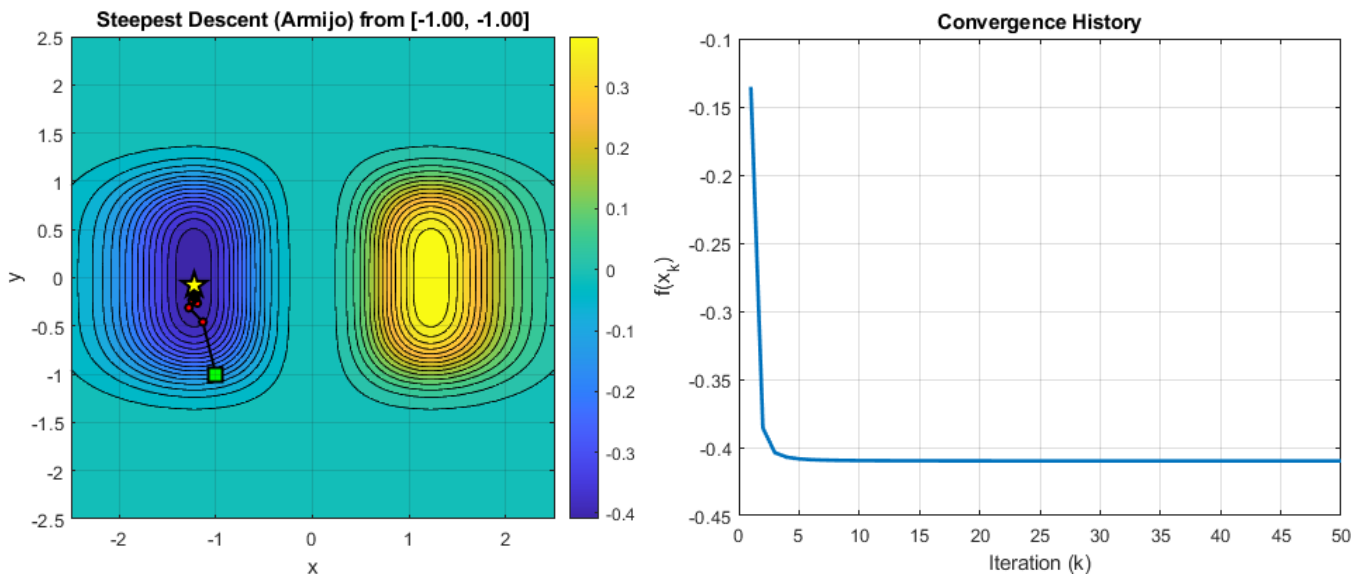


Tuning the step size γ_k

In this section we initialize γ_k with the value 1, as a more conservative choice than the previous large step size.

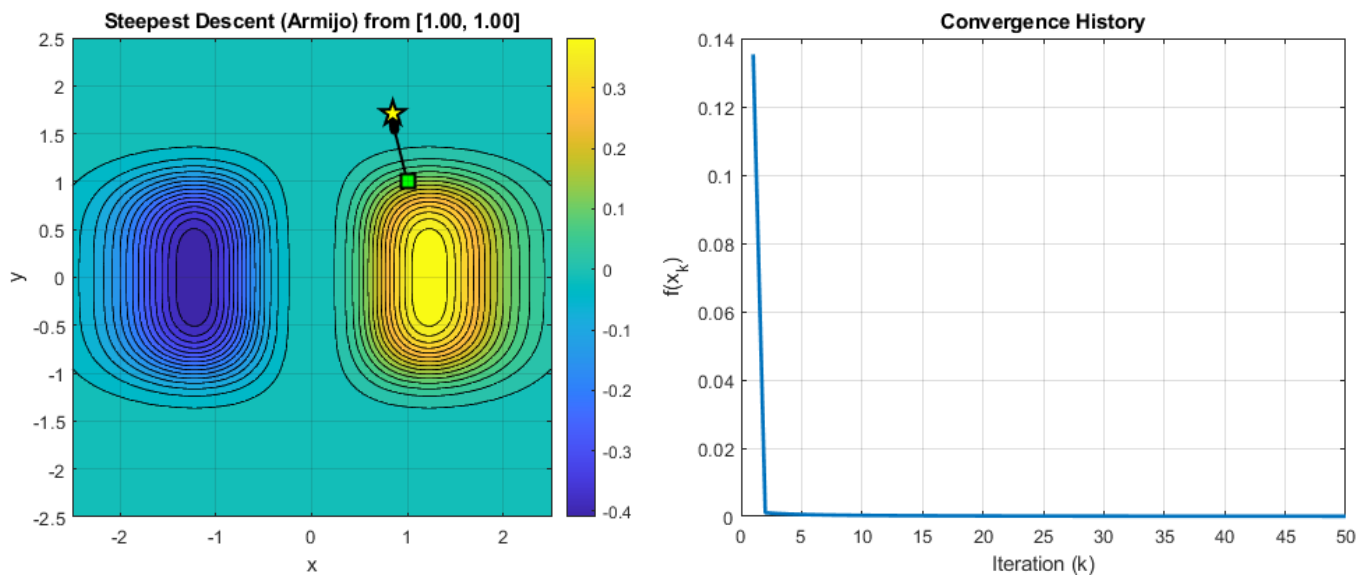
Initial Point $(-1, -1)$

The Euclidean distance between the starting point $(-1, -1)$ and the global minimum $\mathbf{x}^* \approx (-1.22, 0)$ is approximately 1.02. By initializing the backtracking search with $s = 1$, the algorithm attempts a step magnitude that is naturally commensurate with the distance to the solution. This allows the algorithm to cover the majority of the distance in the early iterations without needing to "build up" momentum from small steps. The Armijo condition checks if the decrease in $f(\mathbf{x})$ is sufficient. Because the direction of steepest descent at $(-1, -1)$ points sharply into the valley, a step of size $\gamma = 1$ yields a substantial reduction in the objective function. Consequently, the backtracking loop ($\gamma \leftarrow \beta\gamma$) is rarely triggered in the early phase. The algorithm accepts the large step immediately, minimizing the number of function evaluations per iteration.



Initial Point $(1, 1)$

The gradient at $(1, 1)$ points away from the local maximum peak, directing the algorithm towards increasing y values. Unlike previous experiments with large fixed step sizes that shot the iterator far out of bounds ($y > 2.5$), the Armijo rule effectively regulated the step size. It allowed a large enough step to reduce the function value significantly but kept the iterator within a reasonable numerical range. The convergence history plot shows a sharp, vertical drop in the function value from 0.135 to nearly 0 in the first 2 iterations, followed by a flat line. he algorithm successfully minimized the function locally. By moving to $y \approx 1.7$, the function value became effectively zero, because the term e^{-y^4} dominates and is rendering the gradient magnitude negligible.



Conclusions

The extensive analysis of the Steepest Descent method applied to the function $f(x, y) = x^3 e^{-x^2-y^4}$ elucidates the critical dependency of the algorithm's performance on two fundamental factors: the **topological placement of the initial point** $\mathbf{x}_0$ and the **strategy for selecting the step size** γ_k.

1. Sensitivity to Initialization (Topological Constraints)

Our experiments demonstrate that the convergence of gradient-based methods is not guaranteed globally but is strictly limited to the basin of attraction containing $\mathbf{x}_0$.

- **Basin of Attraction:** When initialized at $(-1, -1)$, the algorithm consistently converged to the global minimum, as the gradient in this region naturally directs the descent into the convex valley.
- **Asymptotic Traps:** Conversely, initialization at $(1, 1)$ revealed a fatal flaw in local search methods. Despite the algorithm functioning correctly (reducing the function value), the topology directed the path towards an asymptotic plateau ($f \to 0$ as $y \to \infty$) rather than the true minimum. This confirms that Steepest Descent is blind to the global landscape and can easily be trapped by trivial solutions in flat regions where $\nabla f \approx \mathbf{0}$.
- **Saddle Points:** The case of $(0, 0)$ confirmed that critical points with zero gradient cause immediate stagnation, requiring external perturbation to restart the descent.

2. Limitations of Fixed Step Sizes

The use of a fixed parameters γ_k proved to be highly unreliable for non-convex functions with varying curvature.

- **Instability:** Large fixed steps (e.g., $\gamma = 2$) resulted in two distinct failure modes: chaotic oscillation across the valley floor (at $(-1, -1)$) and massive overshooting into the zero-gradient region (at $(1, 1)$).
- **Inefficiency:** While small fixed steps ensure stability, they suffer from prohibitively slow convergence rates, making them impractical for high-precision applications.

3. Comparative Analysis: Exact Line Search vs Armijo's Rule

The comparison between adaptive strategies reveals a trade-off between per-step optimality and computational robustness.

Exact Line Search (Derivative-Based Bisection):

- **Convergence Speed:** This method demonstrated the fastest convergence rate in terms of iteration count. By locating the precise minimum along the search direction, it exhausted the descent potential of each step.
- **Trajectory:** The characteristic "zig-zag" oscillation of the step size γ_k (observed between 0.9 and 2.0) is a direct consequence of the orthogonality of consecutive search directions. This allows the algorithm to navigate narrow "canyons" efficiently.

Armijo Rule (Backtracking Line Search):

- **Robustness:** The Armijo rule proved to be the safer strategy. It successfully prevented the algorithm from stepping out of bounds by detecting when a large trial step failed to yield a sufficient decrease.
- **Parameter Dependency:** The performance is sensitive to the hyperparameters s (initial step), β (reduction factor), and σ (acceptance threshold). While it may require more iterations than the Exact Search to reach high precision, it avoids the computational expense of solving a sub-optimization problem at every step.

Conclusion: We conclude that for ill-conditioned objective functions, fixed step sizes should be avoided. While Exact Line Search offers superior convergence rates per iteration, the Armijo Rule provides a crucial layer of stability, making it a generally preferred choice for general-purpose optimization where the function's curvature is unknown.

Requirement 3 - Newton's Method

Newton's Method is a second-order optimization algorithm that utilizes curvature information to accelerate convergence. The iterative update rule is defined as:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \gamma_k \cdot \mathbf{d}_k$$

where the search direction $\mathbf{d}_k$ is determined by solving the Newton system:

$$\mathbf{d}_k = -\left[\nabla^2 f(\mathbf{x}_k)\right]^{-1} \nabla f(\mathbf{x}_k)$$

Here, $\nabla^2 f(\mathbf{x}_k)$ represents the Hessian matrix of second-order partial derivatives.

Convergence Condition: Positive Definiteness

For the search direction $\mathbf{d}_k$ to guarantee a descent (i.e., to reduce the function value) and for the quadratic approximation to model a "valley" rather than a "hill" or "saddle," the Hessian matrix $\nabla^2 f(\mathbf{x}_k)$ must be **Positive Definite** at every iteration step.

- Mathematically, a matrix H is positive definite if $\mathbf{v}^T H \mathbf{v} > 0$ for all non-zero vectors $\mathbf{v}$.
- In terms of spectral analysis, this requires that **all eigenvalues** of the Hessian matrix are strictly positive ($\lambda_i > 0, \forall i$).

Implementation Details

For the implementation of the method, we adhered to Algorithm 5.2.2 (p. 126) of the reference textbook. Our specific deviation from the standard algorithm lies in the selection rule for the step size γ_k. As with the Steepest Descent method, we employ three distinct strategies (Fixed, Exact, Armijo) to determine γ_k.

Eigenvalue Check Mechanism

To ensure the stability of the algorithm, we have integrated a rigorous check for positive definiteness at each iteration point $\mathbf{x}_k$:

1. Calculate the Hessian matrix $H_k = \nabla^2 f(\mathbf{x}_k)$.

2. Compute the eigenvalues of H_k.

3. **Termination Criteria:** If any eigenvalue is found to be non-positive ($\lambda \leq 0$), the algorithm terminates immediately. The procedure returns only the sequence of points $\mathbf{x}_k$ generated up to that moment. The following analysis continues even if any eigenvalue is found to be non-positive, to have a visual representation of the algorithm's divergence trajectory path. Thus, we uploaded 2 different .m files for both cases.

Constant step size γ_k

In the context of Newton's method, the step size γ_k plays a distinct role compared to steepest descent. While the classical Newton method inherently assumes a unit step size ($\gamma_k = 1$) to achieve quadratic convergence, this is strictly valid only when the starting point is sufficiently close to the minimum. In this section, we treat γ_k as a fixed constant to modulate the update magnitude. A reduced step size ($\gamma_k < 1$) acts as a stabilizer in regions where the quadratic approximation is poor, whereas a full step ($\gamma_k = 1$) attempts to fully exploit second-order curvature information for maximum speed.

Case 1: $\gamma_k = 0.2$

Initial Point $(-1, -1)$

The divergent behavior of Newton's Method initialized at $\mathbf{x}_0 = (-1, -1)$ is a deterministic consequence of the local second-order geometry of the objective function. Newton's method calculates the search direction $\mathbf{d}_k = -[\nabla^2 f(\mathbf{x}_k)]^{-1}\nabla f(\mathbf{x}_k)$ based on the assumption that the local curvature can be approximated by a convex quadratic form (a bowl). For this assumption to yield a direction that minimizes the function, the Hessian matrix $\nabla^2 f(\mathbf{x}_k)$ must be Positive Definite (possessing strictly positive eigenvalues). However, at the specific point $(-1, -1)$, the Hessian matrix is Indefinite, meaning it possesses both positive and negative eigenvalues. This implies the local geometry resembles a saddle point. Consequently, the Newton update incorrectly interprets the negative curvature as a direction to traverse, effectively turning the minimization problem into a maximization problem along the eigenvector associated with the negative eigenvalue. This forces the algorithm to move "uphill" away from the local minimum and towards the asymptotic plateau where $f \approx 0$, resulting in the observed increase in the objective function value. To prove the matrix is indefinite, we calculate the second-order partial derivatives of $f(x, y) = x^3 e^{-x^2-y^4}$.1. First Derivatives (Gradient):

$$f_x = x^2(3 - 2x^2)e^{-x^2-y^4}$$

$$f_y = -4x^3y^3e^{-x^2-y^4}$$

2. Second Derivatives (Hessian Elements):
Term f_{xx}:Differentiating f_x with respect to x using the product rule:

$$f_{xx} = \frac{\partial}{\partial x}[(3x^2 - 2x^4)e^{-x^2-y^4}] = (6x - 8x^3)e^{-x^2-y^4} + (3x^2 - 2x^4)(-2x)e^{-x^2-y^4} \& = xe^{-x^2-y^4}[(6 - 8x^2) - 2x^2($$

Term f_{yy}:Differentiating f_y with respect to y:

$$f_{yy} = \frac{\partial}{\partial y}[-4x^3y^3e^{-x^2-y^4}] \& = -4x^3[3y^2e^{-x^2-y^4} + y^3(-4y^3)e^{-x^2-y^4}] = -4x^3y^2e^{-x^2-y^4}(3 - 4y^4)$$

Term f_{xy} : Differentiating f_y with respect to x:

$$f_{xy} = \frac{\partial}{\partial x}[-4x^3y^3e^{-x^2-y^4}]\& = -4y^3e^{-x^2-y^4}[3x^2 + x^3(-2x)] = -4x^2y^3e^{-x^2-y^4}(3-2x^2)$$

3. Evaluation at $\mathbf{x}_0 = (-1,-1)$:Let $E = e^{-(-1)^2-(-1)^4} = e^{-2}.f_{xx}(-1,-1)$:

$$(-1)E(4(1) - 14(1) + 6) = -1 \cdot E \cdot (-4) = \mathbf{4e^{-2}}$$

$f_{yy}(-1,-1)$:

$$-4(-1)^3(-1)^2E(3-4(1)) = -4(-1)(1)E(-1) = \mathbf{-4e^{-2}}$$

$f_{xy}(-1,-1)$:

$$-4(-1)^2(-1)^3E(3-2(1)) = -4(1)(-1)E(1) = \mathbf{4e^{-2}}$$

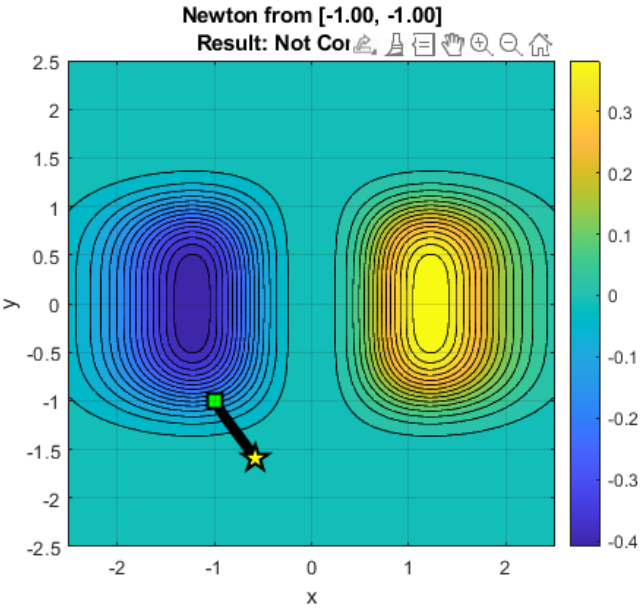
4. The Hessian Matrix:

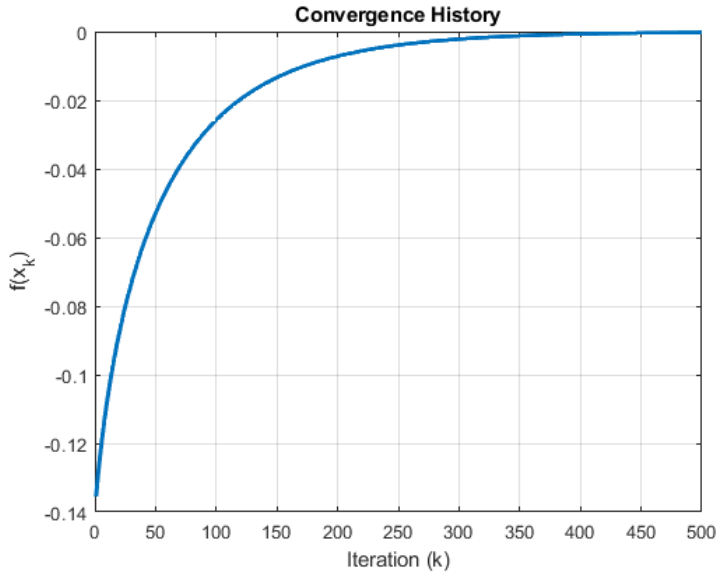
$$H(-1,-1) = e^{-2}\begin{pmatrix} 4 & 4 \\ 4 & -4 \end{pmatrix}$$

5. Eigenvalue Analysis: The determinant of this matrix is:

$$\det(H) = (4)(-4) - (4)(4) = -16 - 16 = -32$$

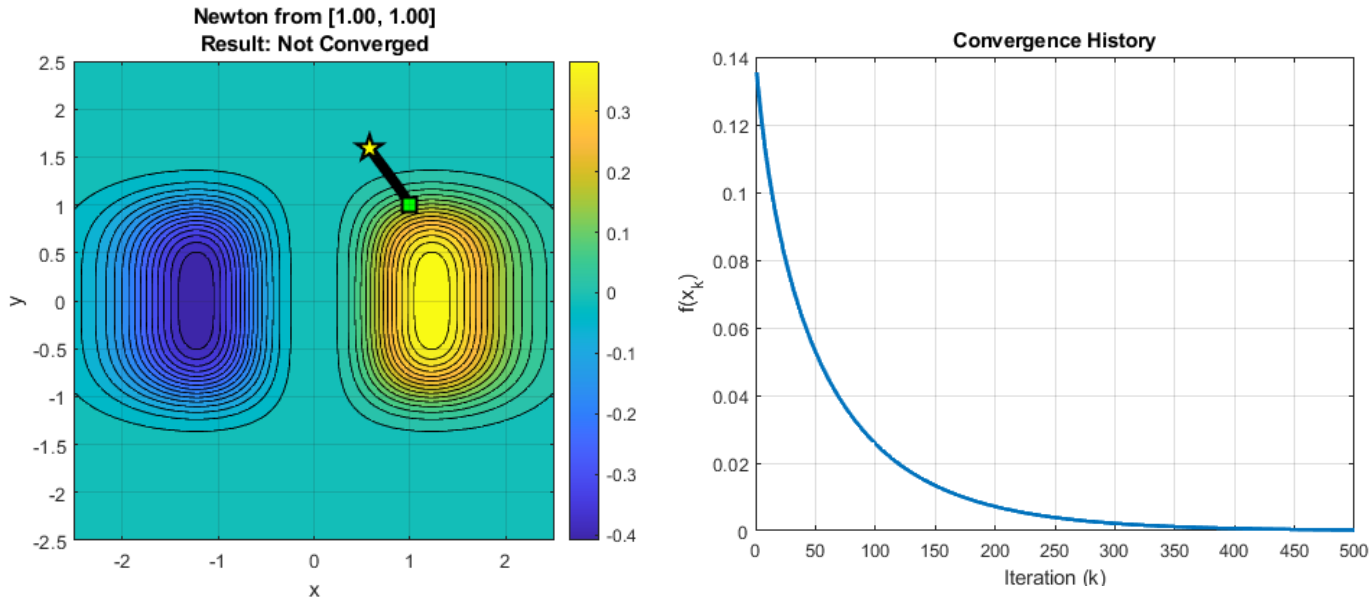
Since the determinant is negative, the eigenvalues must have opposite signs (one positive, one negative).





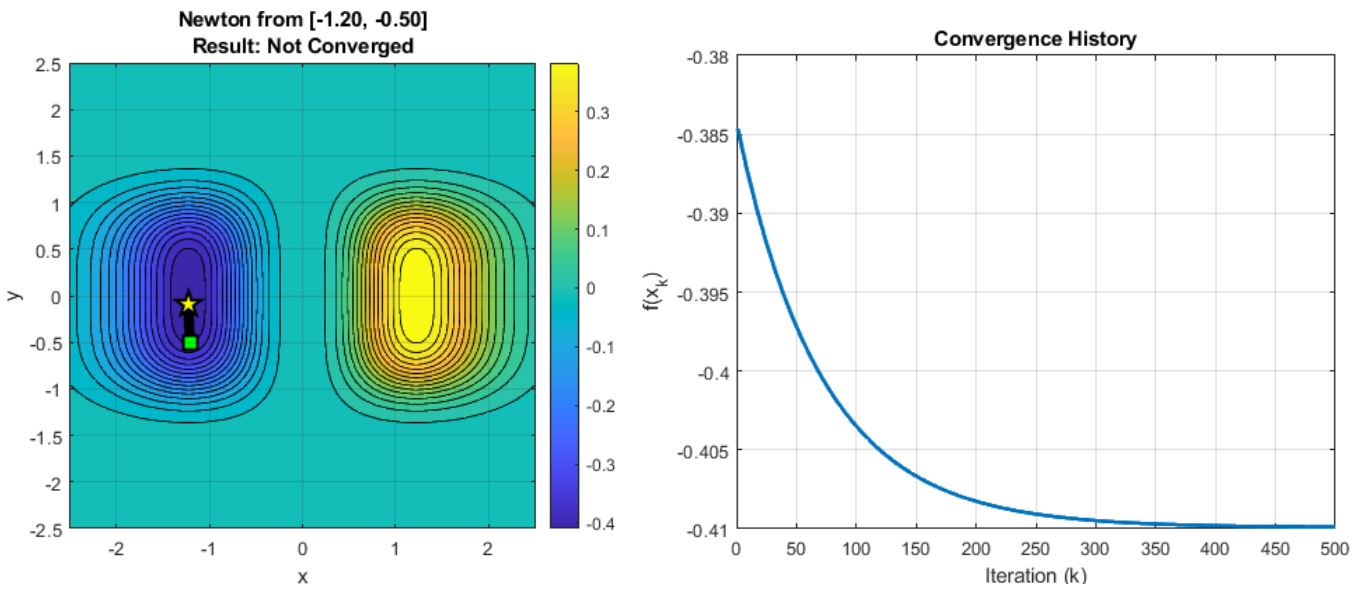
Initial Point $(1,1)$

The application of Newton's Method ($\gamma_k = 0.2$) from the initialization point $(1,1)$ results in a smooth but incorrect convergence trajectory. Instead of locating the global minimum ($f \approx -0.41$), the algorithm minimizes the function by finding the asymptotic region where $f \to 0$.The algorithm is slowly crawling through the region where the function gets flatter and flatter. As $f(x,y)$ approaches zero due to the e^{-y^4} term, the gradient and curvature also vanish. The step sizes become tiny, and the algorithm essentially "drifts" until it hits the iteration limit max=500.



Initial Point $(-1.2, -0.5)$

The simulation initialized at $\mathbf{x}_0 = (-1.2, -0.5)$ with a fixed $\gamma_k = 0.2$ demonstrates correct directional movement and convergence. Unlike the previous starting points, this coordinate lies within the local convex basin of the global minimum. Consequently, the Hessian matrix $\nabla^2 f(\mathbf{x})$ is Positive Definite, and the Newton direction $\mathbf{d}_k$ correctly points towards the minimizer at $(-1.22, 0)$.



Optimal Step Size γ_k minimizing $f(x_k + \gamma_k \cdot d_k)$

In each iteration of the Newton's Method algorithm—after determining the search direction $d_k = -\nabla f(x_k)$—we proceed to calculate the optimal step size γ_k. This process, known as **Exact Line Search**, involves finding the value of γ that minimizes the scalar objective function along

the search direction:

$$g(\gamma) = f(x_k + \gamma \cdot d_k)$$

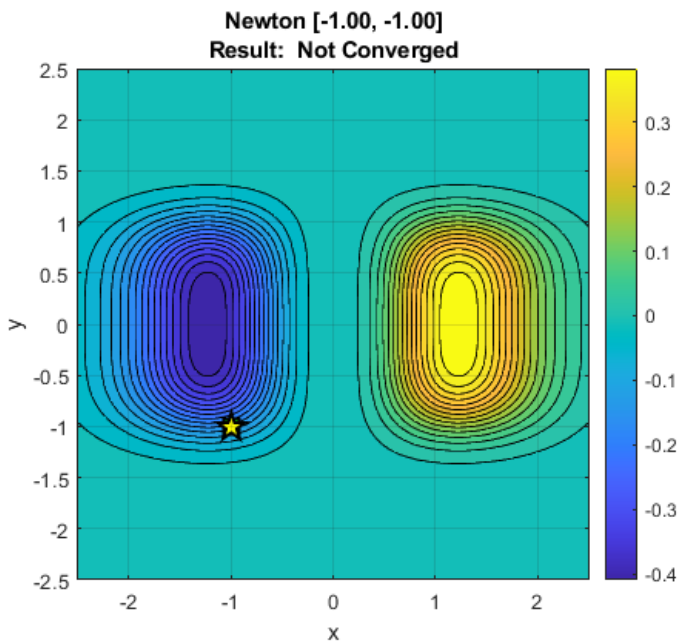
A necessary condition for γ^* to be a local minimizer of $g(\gamma)$ is that the first derivative of g with respect to γ must be zero. By applying the chain rule, this condition is expressed as:

$$g'(\gamma) = \nabla f(x_k + \gamma \cdot d_k)^T \cdot d_k = 0 \tag{3}$$

Consequently, the optimization problem transforms into a root-finding problem for the function $g'(\gamma)$ where γ is chosen to belong in the $(0, 2)$ interval.

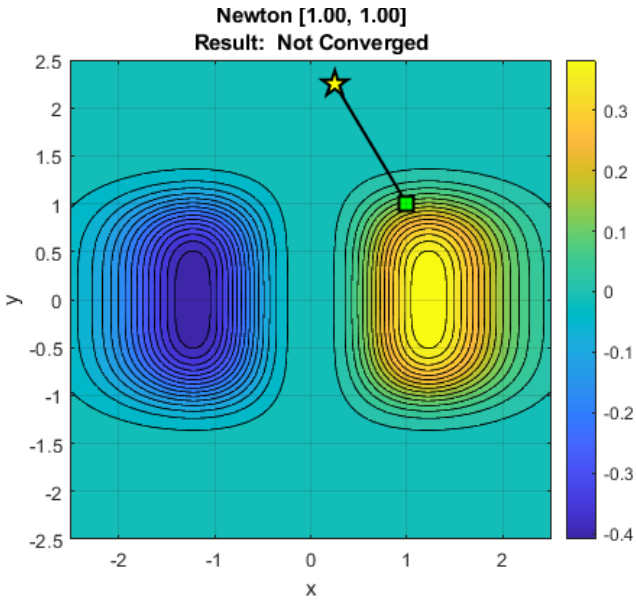
Initial Point $(-1, -1)$

The application of Newton's Method initialized at $\mathbf{x}_0 = (-1, -1)$, utilizing an Exact Line Search to determine the optimal step size γ_k, results in immediate stagnation. Despite the starting point lying visually close to the basin of the global minimum, the algorithm fails to generate a viable descent step. This failure is fundamentally rooted in the spectral properties of the Hessian matrix at the initial point.As established in previous analyses, the Hessian matrix $\nabla^2 f(-1, -1)$ is Indefinite (possessing one positive and one negative eigenvalue). In the context of Newton's method, this implies that the local quadratic approximation of the function resembles a saddle geometry rather than a convex valley. Consequently, the Newton direction $\mathbf{d}_k = -[\nabla^2 f]^{-1} \nabla f$ is not a descent direction; instead, it points towards the saddle's ridge. When the Exact Line Search attempts to minimize the function along this erroneous direction:
It searches for a γ such that $f(\mathbf{x}_0 + \gamma \mathbf{d}_0) < f(\mathbf{x}_0)$. Because $\mathbf{d}_0$ is an ascent direction (pointing uphill or towards a saddle), any positive movement increases the function value. The only "step" that doesn't increase the value is $\gamma = 0$. Thus, the line search correctly determines that staying put is better than moving in the wrong direction, causing the algorithm to terminate instantly at the start point.



Initial Point $(1, 1)$

The application of Newton's Method initialized at $\mathbf{x}_0 = (1, 1)$, utilizing an optimal step size strategy, exhibits an instantaneous convergence to the asymptotic limit. The algorithm computes a search direction $\mathbf{d}_k$ based on the local indefinite Hessian, which orients the trajectory in the North-West direction (y increasing, x decreasing). The line search subroutine, seeking to minimize the objective function along this vector, identifies that the function value decays monotonically towards zero as the distance increases. Consequently, the algorithm selects a large step length that propels the iterator immediately into the region where $y > 2.2$. At this location, the term e^{-y^4} renders the function value and the gradient magnitude indistinguishable from zero (numerical underflow). Thus, the algorithm terminates at the second iteration ($k = 2$), having found a "flat" spot that satisfies the stationary condition $\nabla f \approx \mathbf{0}$, despite failing to locate the true global minimum.



Step Size γ_k updated by Armijo's Rule

The Armijo Rule is an iterative method used to determine a step size γ_k that guarantees a sufficient decrease in the objective function at each iteration, rather than solving for the exact minimum along the search direction.
This strategy enforces the Sufficient Decrease Condition:

$$f(\mathbf{x}_k + \gamma_k \mathbf{d}_k) \leq f(\mathbf{x}_k) + \sigma \gamma_k \nabla f(\mathbf{x}_k)^T \mathbf{d}_k \tag{4}$$

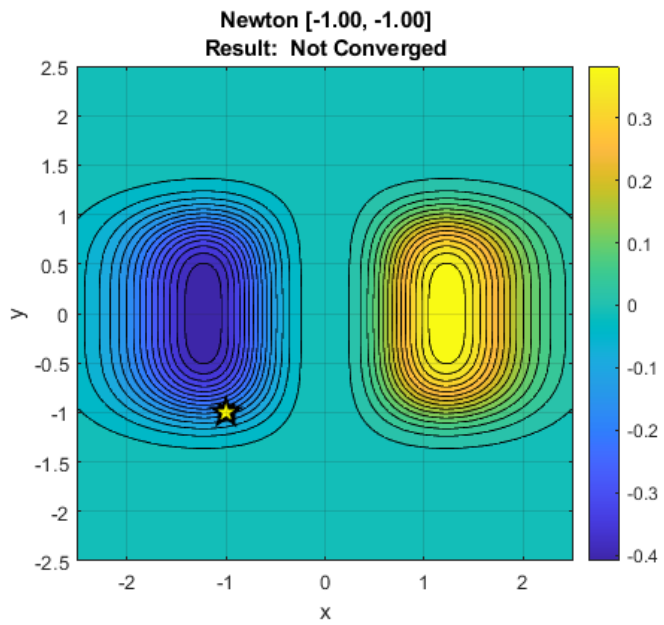
where $\sigma \in (0, 1)$ is a relaxation parameter that defines the minimum acceptable drop in the function value relative to the predicted decrease based on the local gradient. For our project we assigned $\sigma = 10^{-4}$. The algorithm initializes with a trial step length s and iteratively applies a reduction factor $\beta \in (0, 1)$ until the inequality is satisfied. For our project we assigned β=0.2 and an initial large step size value of γ_k=5.
In regions of high curvature, such as the steep walls of the global minimum's basin, the condition rejects large steps that would violate the inequality, forcing the algorithm to "backtrack" to a smaller, stable step size. Conversely, in flatter regions where the gradient magnitude is small, the condition is more easily met, allowing for larger steps. This dynamic adjustment ensures that the algorithm consistently progresses towards the minimum while maintaining strict theoretical stability guarantees.

Initial Point $(-1, -1)$

The application of Newton's Method initialized at $\mathbf{x}_0 = (-1, -1)$ utilizing the Armijo Rule results in an immediate termination of the algorithm without iteration. This failure is structurally identical to the case of Exact Line Search and is caused by the indefinite nature of the Hessian matrix at the starting coordinates. Because $\nabla^2 f(\mathbf{x}_0)$ possesses negative eigenvalues, the computed Newton direction $\mathbf{d}_0 = -[\nabla^2 f]^{-1} \nabla f$ is not a descent direction; mathematically, $\nabla f^T \mathbf{d}_0 > 0$.The Armijo rule requires the trial step to satisfy the sufficient decrease condition:

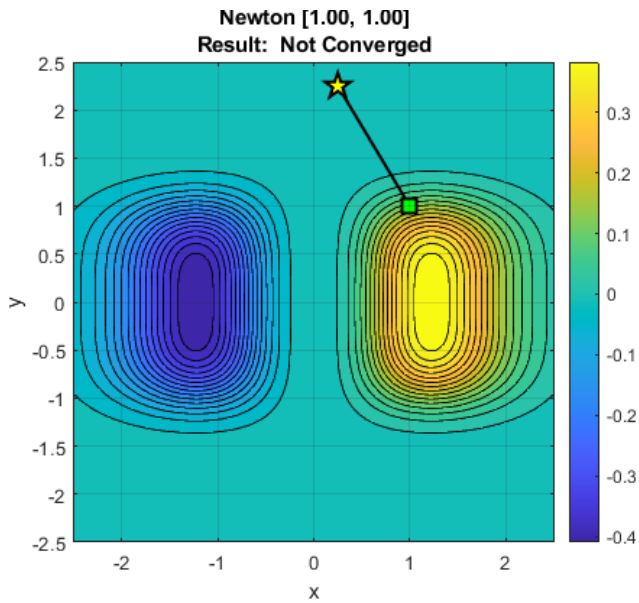
$$f(\mathbf{x_0} + \gamma \mathbf{d_0}) \leq f(\mathbf{x_0}) + \sigma \gamma \nabla f(\mathbf{x_0})^T \mathbf{d_0}$$

Since the directional derivative $\nabla f^T \mathbf{d_0}$ is positive (indicating an ascent), the term on the right-hand side is strictly greater than $f(\mathbf{x}_0)$ (for the typical negative contribution term). However, physically moving in the direction $\mathbf{d}_0$ increases the function value. The backtracking mechanism iteratively reduces γ towards zero, but for any $\gamma > 0$, the inequality fails.



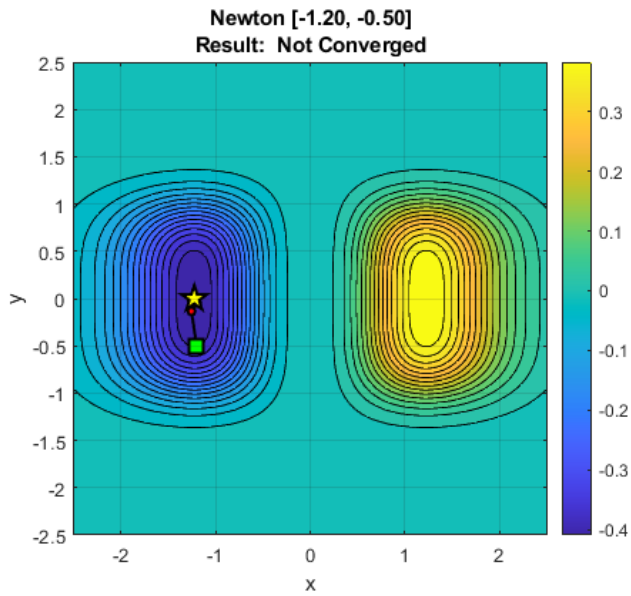
Initial Point $(1, 1)$

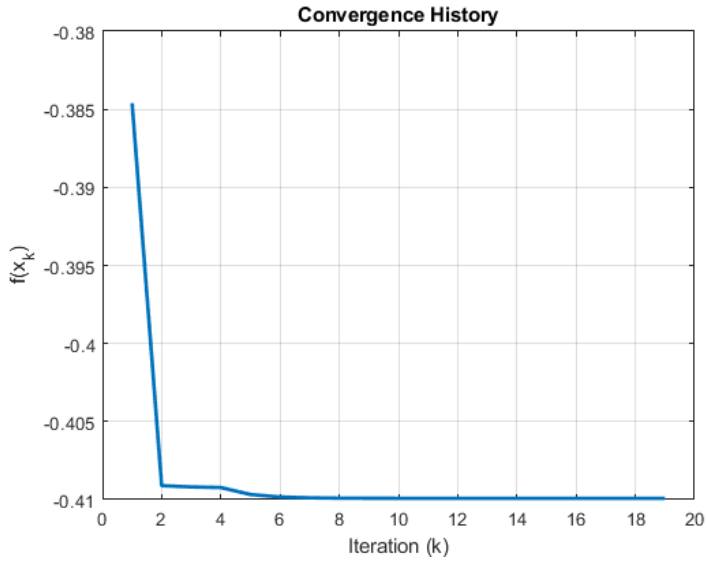
The simulation of Newton's Method initialized at $\mathbf{x}_0 = (1, 1)$ utilizing the Armijo Rule exhibits an instantaneous convergence to the trivial asymptotic limit, mirroring the behavior of the Exact Line Search. The algorithm computes a Newton direction vector determined by the local indefinite curvature, which orients the search towards the North-West. Because the Armijo rule initializes the line search with a unitary trial step ($\gamma = 2$), and given the aggressive exponential decay of the objective function term e^{-y^4}, this full step propels the iterator immediately into the asymptotic region (approximately $y \approx 2.25$) where the function value drops below machine precision. Since this transition yields a substantial decrease in the objective function—satisfying the sufficient decrease condition instantly—the backtracking mechanism is never triggered. Consequently, the algorithm accepts the full Newton step, landing in a region of vanishing gradient ($\nabla f \approx \mathbf{0}$) and terminating at the second iteration ($k = 2$), effectively locating the zero-value plateau rather than the global minimum.

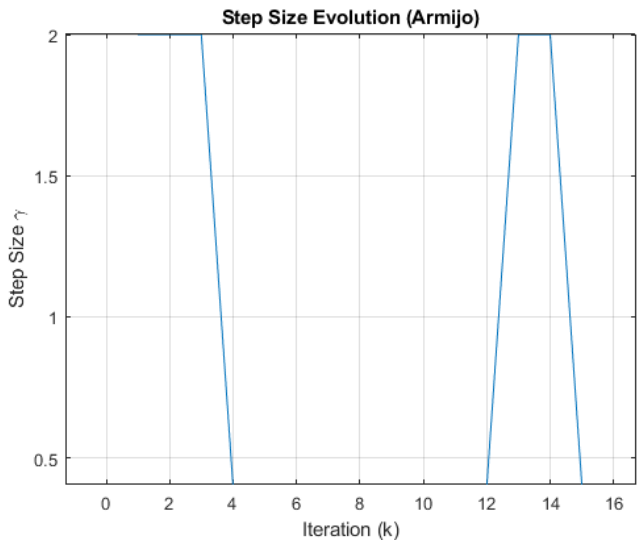


Initial Point $(-1.2, -0.5)$

This test case is crucial because it demonstrates the Local Convergence property of Newton's Method: it works brilliantly, but only when started sufficiently close to the minimum where the function is convex. The simulation initialized at $\mathbf{x}_0 = (-1.2, -0.5)$ provides the first successful convergence of Newton's Method to the global minimum. This success is attributed to the topological placement of the starting point. Unlike $(-1, -1)$, which was located in a saddle region with an indefinite Hessian, the point $(-1.2, -0.5)$ lies deeply within the convex basin of the global minimum. In this region, the Hessian matrix $\nabla^2 f(\mathbf{x})$ is Positive Definite (all eigenvalues are positive). Consequently, the Newton direction $\mathbf{d}_k = -H^{-1}\mathbf{g}$ is a valid and powerful descent direction that points directly towards the bottom of the "bowl". The function value drops precipitously from -0.385 to the global minimum of ≈ -0.41 in fewer than 6 iterations. The step size graph shows the algorithm attempting aggressive steps ($\gamma \approx 2$). Because the local shape is well-modeled by a parabola, the Armijo rule permits these large steps (or standard steps of $\gamma = 2$) without significant backtracking, accelerating the arrival at the solution.







Comments and Conclusions

1. Critical Dependency on Initialization

From the comprehensive analysis of Newton's Method, it is evident that the choice of the initial starting point $\mathbf{x}_0$ plays a decisive role, not only in the speed of convergence but in the fundamental applicability of the algorithm.

- **Basin of Attraction:** Convergence is strictly local. The method successfully converges only when initialized within the convex basin of the minimum (e.g., $\mathbf{x}_0 = [-1.2, -0.5]$).
- **Hessian Condition:** The algorithm's viability is contingent on the spectral properties of the Hessian matrix $\nabla^2 f(\mathbf{x}_0)$. In regions where the Hessian is indefinite (having both positive and negative eigenvalues, as seen at $[-1, -1]$ and $[1, 1]$), the Newton direction is not guaranteed to be a descent direction. In such cases, the algorithm fails to minimize the objective function, often drifting towards saddle points or asymptotic plateaus.

2. Evaluation of Step-Size Strategies

Regarding the specific case where the method successfully converges, we observe distinct trade-offs between the step-size selection rules:

- **Armijo Rule Efficiency:** While the Armijo rule provides a robust guarantee of global convergence (by enforcing the sufficient decrease condition), it may computationally appear "slower" in terms of pure iteration count compared to an ideal Exact Line Search. This is because the Armijo rule accepts a step as soon as it is "good enough," whereas an Exact search expends extra computational effort to find the mathematically optimal step along the vector.
- **Hyperparameter Sensitivity:** However, as noted in the results, this observation is heavily dependent on the specific tuning of the Armijo hyperparameters: the initial trial step s, the reduction factor β, and the acceptance parameter σ.
- **Quadratic Convergence:** It is crucial to note that with well-tuned parameters (specifically initiating with $s = 2$), the Armijo rule successfully enabled the convergence characteristic of Newton's method in 6th iteration, making it the most practical choice for general non-convex problems.

Requirement 4 - Levenberg-Marquardt Method

The Levenberg-Marquardt method serves as a regularization of Newton's method, specifically designed to guarantee global convergence in regions where the Hessian matrix is not positive definite. To determine the next iteration point $\mathbf{x}_{k+1} = \mathbf{x}_k + \gamma_k \mathbf{d}_k$, the method calculates the search direction $\mathbf{d}_k$ by solving the modified linear system $(\nabla^2 f(\mathbf{x}_k) + \mu_k I)\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$, where I is the identity matrix and $\mu_k \geq 0$ is a scalar damping parameter. The primary mathematical objective of this modification is to enforce positive definiteness via a technique known as spectral shifting. By selecting a sufficiently large μ_k, the eigenvalues of the original Hessian are shifted by $+\mu_k$, ensuring that the modified matrix possesses strictly positive eigenvalues. This guarantees that the system is invertible and that the resulting vector $\mathbf{d}_k$ is always a valid descent direction, correcting the trajectory issues observed in the standard Newton method at saddle points.

This regularization parameter μ_k effectively creates a continuous interpolation between two fundamental optimization strategies based on the local topology of the function. When μ_k is close to zero, the modification becomes negligible, and the algorithm behaves identically to Newton's method, exploiting second-order curvature information to achieve rapid quadratic convergence near the minimum. Conversely, when μ_k is large—typically required when the Hessian is indefinite or the current approximation is poor—the diagonal term dominates the matrix structure. In this regime, the update direction approximates $\mathbf{d}_k \approx -\frac{1}{\mu_k}\nabla f(\mathbf{x}_k)$, causing the algorithm to mimic the Steepest Descent method with a conservative step size. This hybrid nature ensures progress when far from the solution while retaining high efficiency when close to the target. For the implementation of this study, we adhered to the logic defined in Algorithm 5.2.3 (page 139) of the book.

Constant step size γ_k

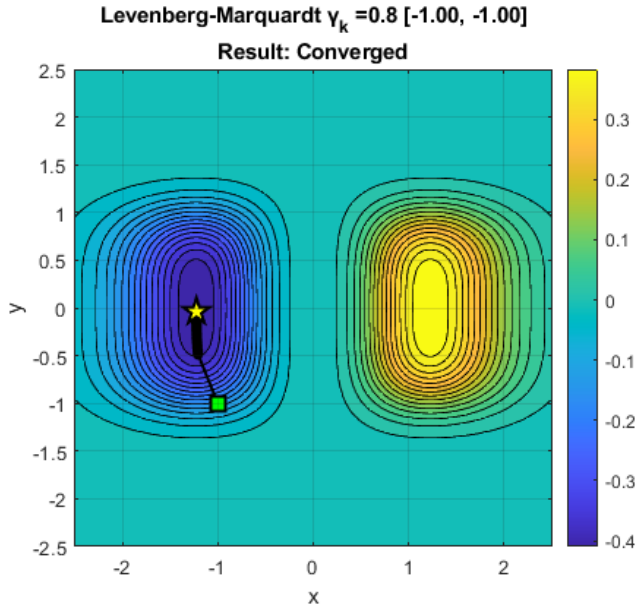
Case 1: γ_k=0.8

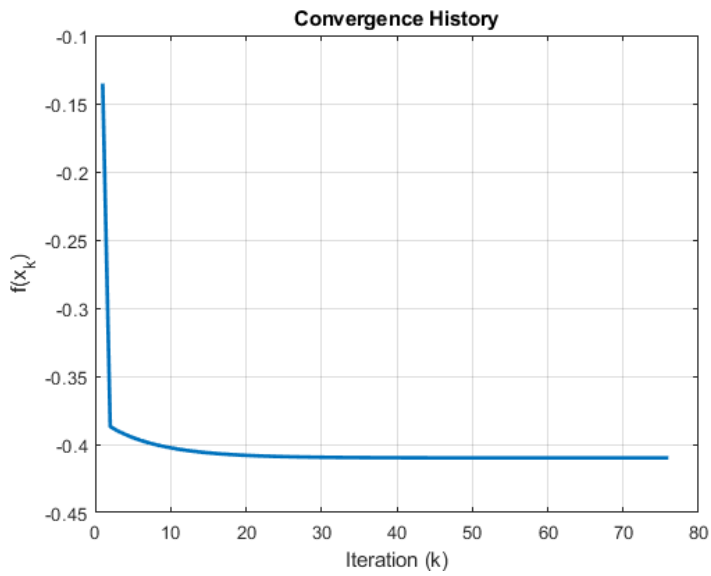
Initial Point $(-1, -1)$

The application of the Levenberg-Marquardt algorithm initialized at $\mathbf{x}_0 = (-1, -1)$ demonstrates a successful convergence to the global minimum, overcoming the spectral deficiencies that caused the standard Newton method to diverge. Recall that at $(-1, -1)$, the Hessian matrix $\nabla^2 f$ is indefinite (it has negative eigenvalues), which caused the standard Newton method to generate an ascent direction towards the asymptotic plateau.The Levenberg-Marquardt method overcomes this by adding a damping term $\mu_k I$ to the Hessian:

$$H_mod = \nabla^2 f(\mathbf{x}_k) + \mu_kI$$

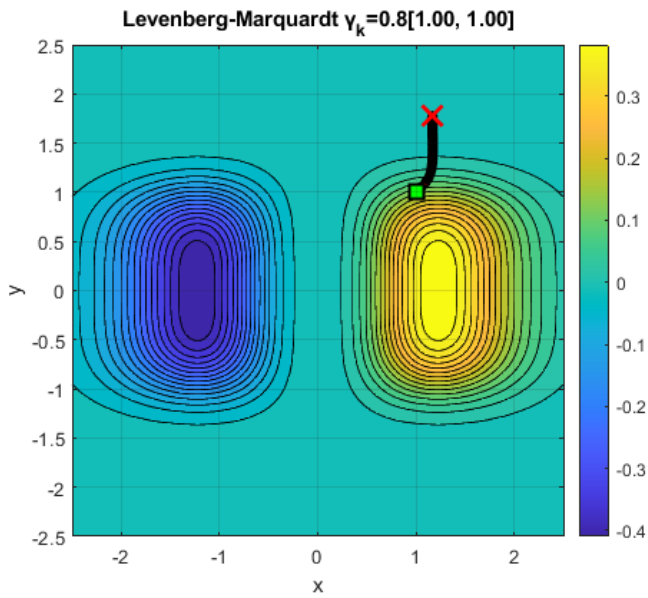
By selecting a sufficiently large μ_k, the algorithm shifts the spectrum of eigenvalues such that H_{mod} becomes Positive Definite. This mathematically guarantees that the computed search direction $\mathbf{d}_k = -H_{mod}^{-1}\nabla f$ is a valid descent direction ($\mathbf{d}_k^T \nabla f < 0$). At the start, where the Hessian is indefinite, μ_k must be large to enforce positive definiteness. As a result, the update direction approximates the negative gradient: $\mathbf{d}_k \approx -\frac{1}{\mu_k}\nabla f$. This effectively forces the algorithm to behave like Steepest Descent, guiding it safely down the slope and away from the saddle point.As the iterator enters the convex basin of the minimum (the blue center), the Hessian naturally becomes positive definite. The damping parameter μ_k can then be reduced, allowing the algorithm to regain Newton-like quadratic convergence for the final approach.

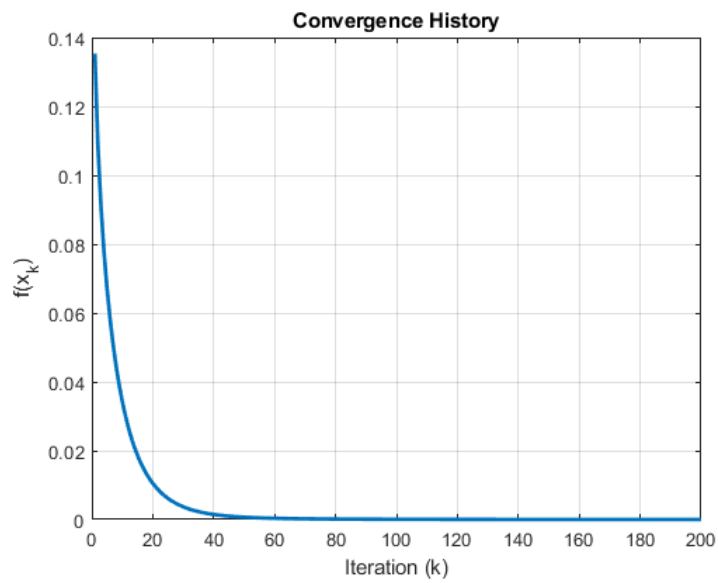




Initial Point $(1, 1)$

The Levenberg-Marquardt algorithm successfully stabilizes the descent direction but fails to locate the global minimum due to topological constraints. Despite this stability, the starting point $(1, 1)$ lies in a region where the negative gradient points towards the "flat" zone ($y \to \infty$). The algorithm efficiently minimizes the function to 0 (as seen in the convergence plot) rather than finding the global minimum of -0.41.



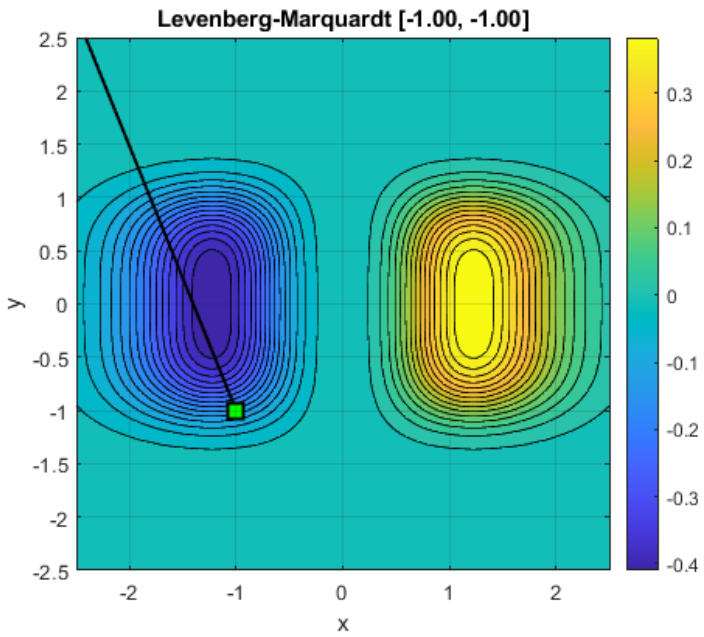


Case 2: γ_k=2.5

This case illustrates a failure mode caused by an excessively aggressive step size, leading to divergence away from the minimum.

Initial Point $(-1, -1)$

The application of the Levenberg-Marquardt algorithm initialized at $\mathbf{x}_0 = (-1, -1)$ with a fixed, large step size of $\gamma = 2.5$ results in immediate divergence from the basin of attraction.The failure is entirely due to the magnitude of the step size $\gamma = 2.5$.The algorithm calculates a direction intended for a step size of roughly $\gamma \approx 1$ (the Newton-like ideal).By forcing a multiplier of 2.5, the algorithm "overshoots" the target minimum significantly. At the new location (far top-left), the function value is very close to zero (due to the e^{-y^4} and e^{-x^2} terms). The algorithm has essentially "escaped" the local features of the function and landed in the asymptotic plateau. It failed to find the specific global minimum well of interest because the step was too large to be contained by the valley's geometry.



Optimal Step Size γ_k minimizing $f(x_k + \gamma_k \cdot d_k)$

In each iteration of the Levenberg-Marquardt's algorithm—after determining the search direction $d_k = -\nabla f(x_k)$—we proceed to calculate the optimal step size γ_k. This process, known as **Exact Line Search**, involves finding the value of γ that minimizes the scalar objective function along the search direction:

$$g(\gamma) = f(x_k + \gamma \cdot d_k)$$

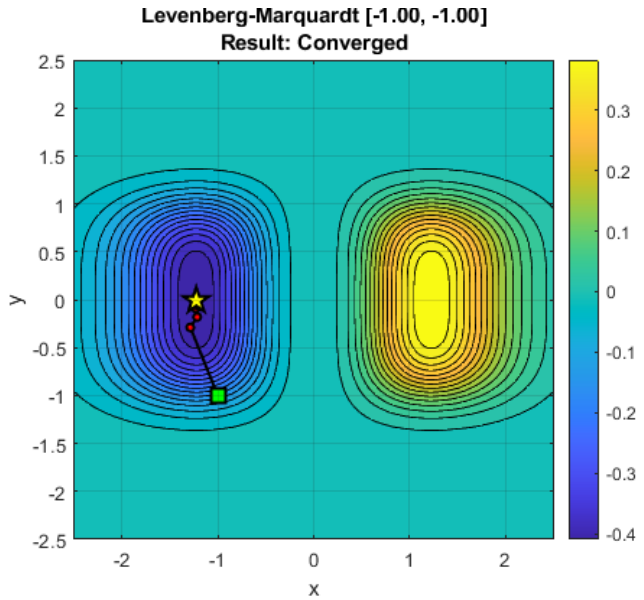
A necessary condition for γ^* to be a local minimizer of $g(\gamma)$ is that the first derivative of g with respect to γ must be zero. By applying the chain rule, this condition is expressed as:

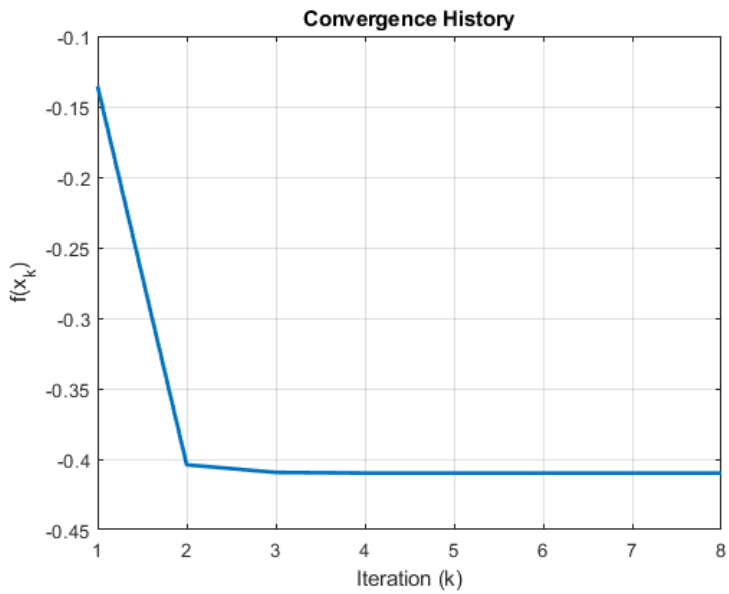
$$g'(\gamma) = \nabla f(x_k + \gamma \cdot d_k)^T \cdot d_k = 0 \tag{5}$$

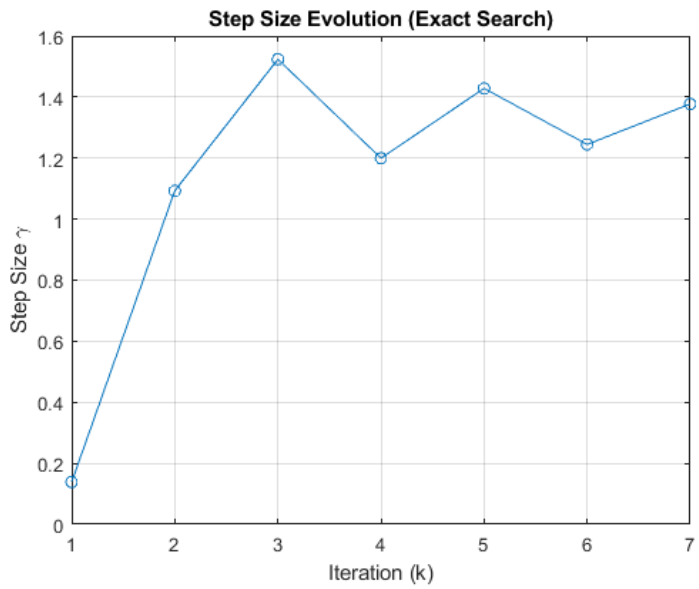
Consequently, the optimization problem transforms into a root-finding problem for the function $g'(\gamma)$ where γ is chosen to belong in the $(0, 2)$ interval.

Initial Point $(-1, -1)$

The application of the Levenberg-Marquardt method coupled with an Exact Line Search strategy from $\mathbf{x}_0 = (-1, -1)$ yields a fast convergence to the global minimum.At the starting point $(-1, -1)$, the standard Newton direction is erroneous due to the indefinite Hessian. The Levenberg-Marquardt regularization ($\mu_k I$) successfully rotates this vector into a valid descent direction.The Convergence History shows a drastic reduction in error. The function value drops from -0.14 to the global minimum floor (≈ -0.41) in effectively 2 to 3 iterations.The step size plot reveals an adaptive behaviour.The Exact Line Search determined that a small fraction of this vector ($\gamma \approx 0.15$)was optimal to reach the valley floor.As the iterator entered the convex basin, the direction became more Newton-like. The step sizes varying around $1.2 - 1.5$ indicate that the Exact Line Search found it beneficial to stretch the Newton step slightly to reach the precise minimum faster.

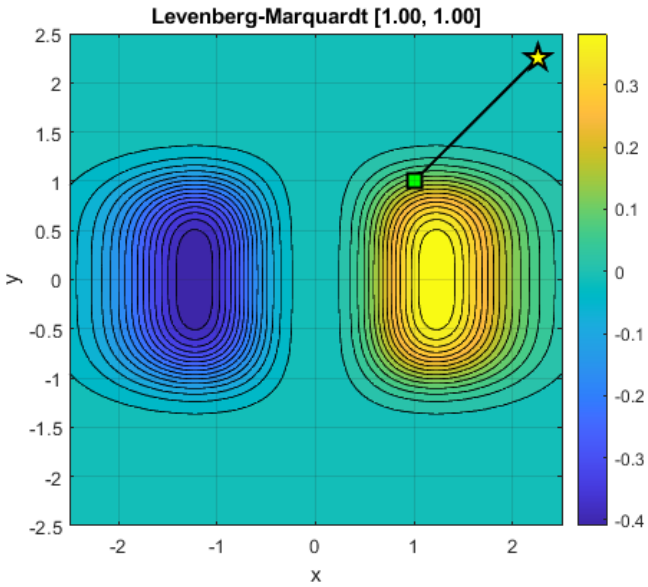


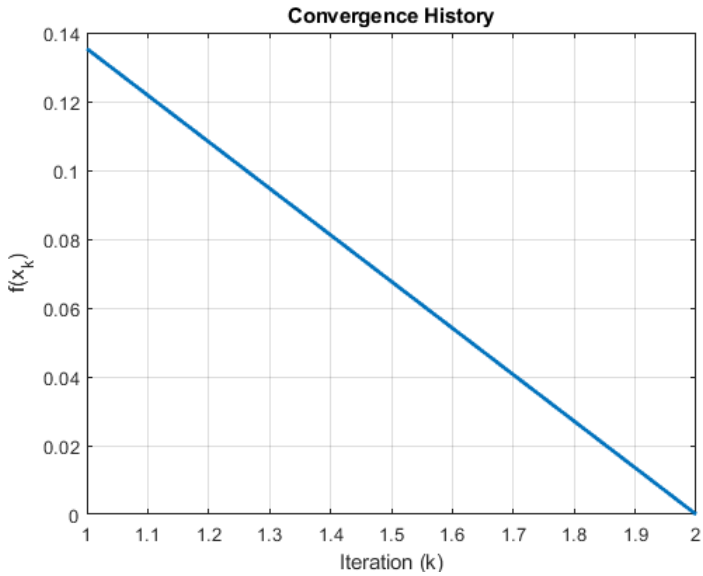




Initial Point $(1, 1)$

The simulation of the Levenberg-Marquardt method utilizing an Exact Line Search strategy from the initialization point $\mathbf{x}_0 = (1, 1)$ exhibits a transition to the trivial asymptotic limit of the objective function. Although the Levenberg-Marquardt regularization corrects the indefinite Hessian to yield a valid descent direction (and even got the algorithm running), the local topology at the starting coordinate orients this vector away from the global minimum. Consequently, the Exact Line Search subroutine, which seeks to minimize the objective function globally along this ray, identifies that the function decays monotonically towards zero as the distance increases. This results in the calculation of a substantial step magnitude that propels the iterator immediately into the region of exponential decay (approximately $y \approx 2.25$). The algorithm thus terminates at the second iteration due to the vanishing gradient, having efficiently located the zero-value plateau but failing to cross the topological ridge required to reach the true global minimizer.





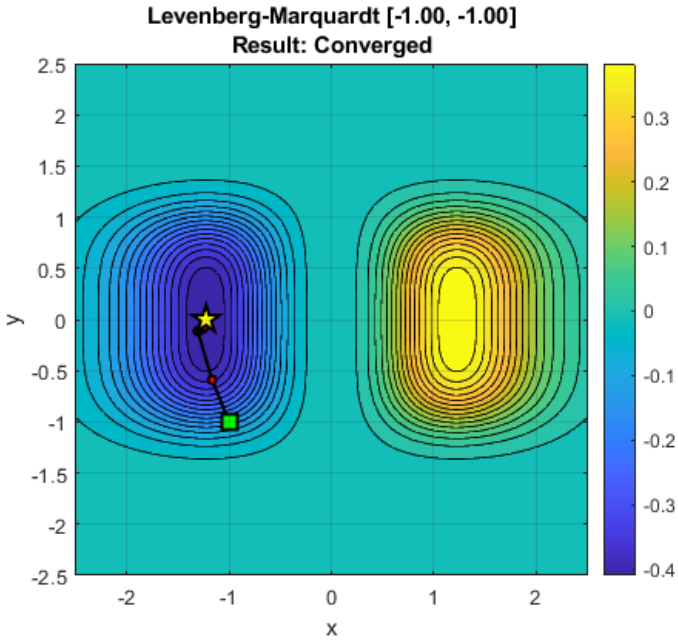
Step Size γ_k updated by Armijo's Rule

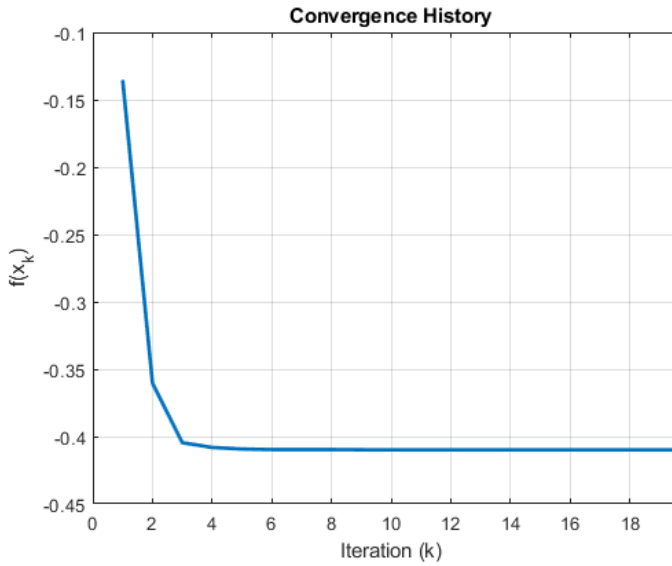
The integration of the Armijo Rule into the Levenberg-Marquardt algorithm decouples the regularization of the search direction from the regulation of the step magnitude. In standard Levenberg-Marquardt implementations, the damping parameter μ_k often controls both the direction and the step length simultaneously. However, by treating the Levenberg-Marquardt update as a distinct line search method, we first compute a search direction $\mathbf{d}_k$ by solving the regularized system $(\nabla^2 f(\mathbf{x}_k) + \mu_k I)\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$. This ensures that $\mathbf{d}_k$ is a valid descent vector even in regions of negative curvature. Subsequently, the Armijo Rule determines the scalar step size γ_k along this specific vector. Starting with a unitary trial step ($\gamma = 2$), the algorithm backtracks if the sufficient decrease condition $f(\mathbf{x}_k + \gamma_k \mathbf{d}_k) \leq f(\mathbf{x}_k) + \sigma\gamma_k \nabla f(\mathbf{x}_k)^T \mathbf{d}_k$ is violated. This hybrid strategy allows the algorithm to exploit the curvature-corrected direction of this method while preventing the overshooting or instability that can occur.

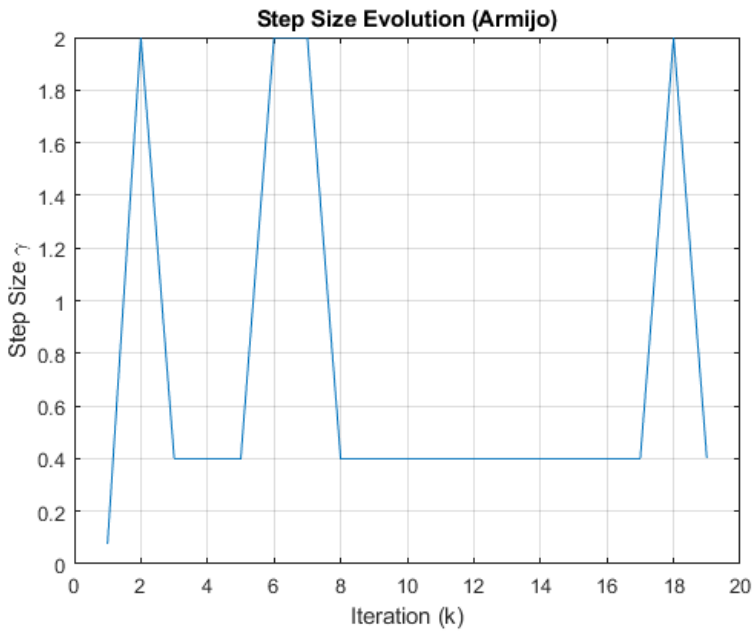
- **Initial Step Size ($\gamma_{init} = 2$):** The search initiates with a relatively aggressive step size. This choice allows the algorithm to take large strides in regions of low curvature, potentially accelerating convergence when far from the minimum.
- **Contraction Factor ($\beta = 0.2$):** When the sufficient decrease condition is not met, the step size is reduced significantly at each backtracking iteration ($\gamma_{new} = 0.2 \cdot \gamma_{old}$). A value of 0.2 implies an aggressive reduction, prioritizing stability by quickly shrinking the step to a safe magnitude in steep regions.
- **Acceptance Parameter ($\sigma = 10^{-4}$):** A standard small value ensures that the algorithm accepts any step that provides a decrease slightly better than zero, enforcing the descent property without being overly restrictive.

Initial Point $(-1, -1)$

The application of the Levenberg-Marquardt algorithm coupled with the Armijo Rule for step size selection demonstrates a stable convergence to the global minimum from the initialization point $\mathbf{x}_0 = (-1, -1)$.At the start point $(-1, -1)$, the Hessian is indefinite. The Levenberg-Marquardt regularization ($\mu_k I$) corrects this, generating a valid descent direction. The Step Size Evolution graph offers an insight into the Armijo mechanism:
The spikes indicate that the algorithm frequently attempted the aggressive initial step ($\gamma_{init} = 2$). In regions where the quadratic model (regularized by LM) was accurate, this large step was accepted, accelerating convergence.The drops indicate iterations where the large step violated the sufficient decrease condition. The algorithm "backtracked" to find a safe step size of 0.4. This confirms that the Armijo rule actively prevented overshooting in steep regions. The Convergence History shows that the objective function value stabilizes at the global minimum ($f^* \approx -0.413$) in fewer than 4 iterations.

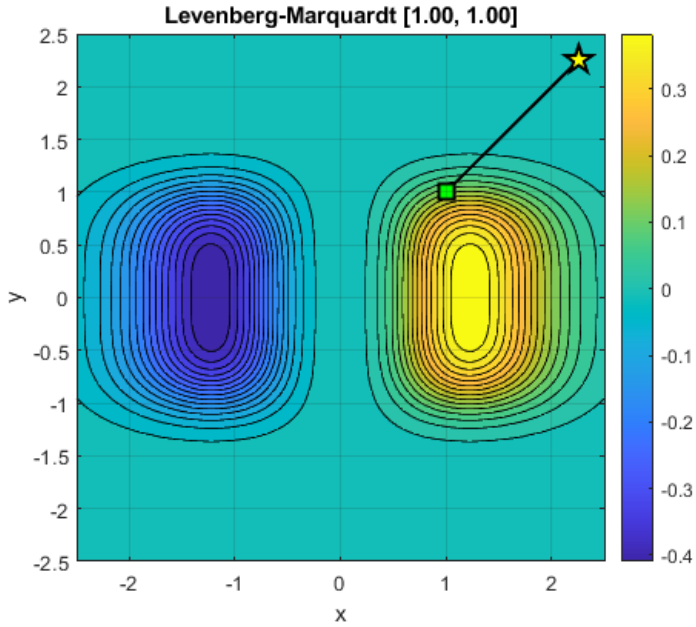


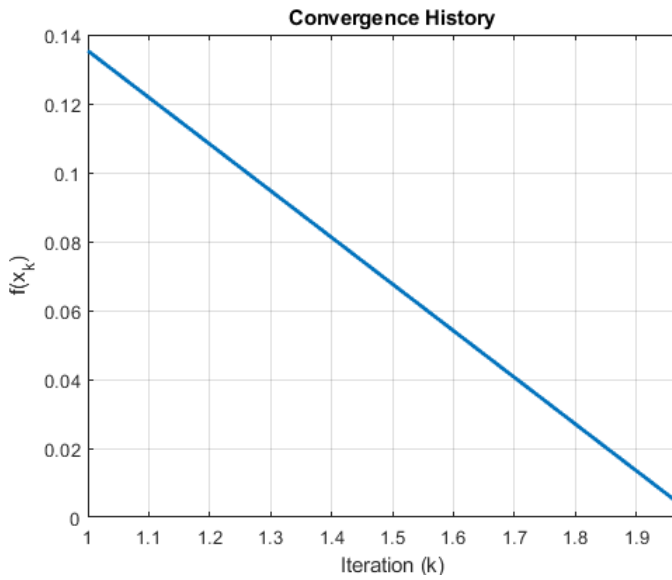




Initial Point $(1, 1)$

At the starting point $(1, 1)$, the Hessian is indefinite. The Levenberg-Marquardt regularization term $\mu_k I$ successfully corrects this, making the matrix positive definite and ensuring a descent direction. However, due to the local topology (being on the slope of a local hill), this "correct" descent direction points North-East, away from the global minimum and towards the open region where $y \to \infty$. The Armijo line search likely initiates with a trial step of $\gamma = 1$. The algorithm tests this full step. Because the objective function contains the term e^{-y^4}, the function value decays precipitously along this path. The candidate point (at approximately $2.3, 2.3$) has a function value of effectively zero. Since $0 \ll 0.135$ (the initial value), the Sufficient Decrease Condition is satisfied immediately. The algorithm accepts this large step without any need to backtrack.At this new coordinate, the gradient is zero, causing the algorithm to terminate.





Conclusions

From the comprehensive analysis of the Levenberg-Marquardt algorithm, we conclude that both the initialization point and the step-size selection rule γ_k play a decisive role in the convergence behavior.

Step Size Analysis

In summary, we deduce that using a **constant step size** γ_k is rarely optimal; while it may allow for convergence in benign regions, it introduces instability and inefficiency in areas of high curvature. We must always account for the topological distance of the starting point from the minimum and the existence of other stationary points ($\nabla f = 0$).

Regarding the adaptive strategies:

- The **Exact Line Search** appears to converge faster in terms of total iterations, as it exhausts the descent potential of the direction vector at every step.
- The **Armijo Rule**, while potentially requiring more steps to reach high precision, offers a robust safety guarantee. However, its efficiency is strictly coupled to the hyperparameter tuning (the initial step s, acceptance threshold α, and reduction factor β).

The Critical Role of the Damping Parameter μ_k

The regularization parameter μ_k is the defining mechanism that governs the algorithm's hybrid nature:

- **For small values of μ_k:** The influence of the term $\mu_k I$ is negligible, and the Hessian information dominates. In this regime, the algorithm exhibits characteristics similar to **Newton's Method**, achieving rapid quadratic convergence when close to the minimum.
- **For large values of μ_k:** The term $\mu_k I$ dominates the Hessian ($\mu_k I \gg \nabla^2 f$). This makes the matrix diagonally dominant, and the search direction approximates $\mathbf{d}_k \approx -\frac{1}{\mu_k}\nabla f$.

Consequently, the algorithm mimics **Steepest Descent**, ensuring stability and a valid descent direction even when the Hessian is indefinite (as observed at $\mathbf{x}_0 = (-1, -1)$).

The dynamic adjustment of μ_k is therefore critical, as it allows the algorithm to seamlessly transition between the stability of Gradient Descent and the speed of Newton's method.

Final Comparison

Ultimately, the Levenberg-Marquardt method demonstrates superior performance compared to the Steepest Descent method. It converges significantly faster and resolves the spectral issues (indefinite Hessian) that caused the standard Newton method to fail. This aligns with theoretical expectations, as the method effectively combines second-order curvature information with gradient-based stability.